SYSTEM DESIGN

INTERVIEW NOTES

25 Essential Topics · SDE-2 Level Depth

Based on: ByteByteGo System Design Interview Course

■ Requirements → Estimation → Design → Deep Dive → Trade-offs

■ Interview-ready answers for every topic

■ ASCII diagrams + schema + API design included

■ Bottlenecks, scaling, and production considerations

■ Mnemonics and memory hooks for each topic

TABLE OF CONTENTS

1 Rate Limiter

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Controls how many requests a client can make in a time window

• Real-world: Twitter API (300 tweets/3hr), Stripe (100 req/sec), AWS APIs

• Prevents: DoS attacks, resource exhaustion, cost overruns

• ■ "Rate limiting is about protecting system availability"

B. REQUIREMENTS

FUNCTIONAL:

• Limit requests per user/IP/API key within time window

• Return HTTP 429 when limit exceeded

• Support multiple rate limit rules (per endpoint, per user tier)

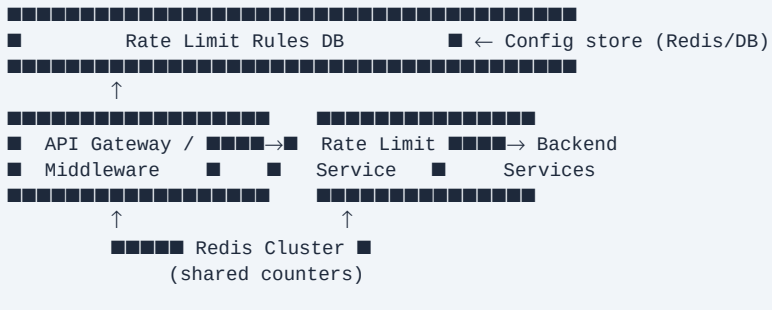
• Rules configurable without code deploy

NON-FUNCTIONAL:

• Low latency (<5ms added overhead)

```
• Highly available (failure → fail open or closed?)
• Distributed (shared state across servers)
• Scale: 10M users, 100K req/sec peak
```

C. CORE COMPONENTS



D. HIGH-LEVEL DESIGN

```
Client → [LB] → [API Gateway + Rate Limiter] → [Backend]
                          ↕
                  [Redis Cluster]
                          ↕
                  [Rules Cache]

Flow:
1. Request arrives → extract user_id/IP
2. Check Redis counter for this window
3. If count < limit → increment + allow
4. If count >= limit → return 429
5. Set X-RateLimit-Remaining, X-RateLimit-Reset headers
```

E. ALGORITHMS (DEEP DIVE)

1. TOKEN BUCKET ■ (most common interview answer)

• Bucket holds N tokens, refills at rate R/sec

• Each request consumes 1 token

• Burst allowed up to bucket size

```
   • Memory: O(1) per user

2. LEAKY BUCKET
   • FIFO queue, processes at fixed rate
   • Smooths traffic, no burst
   • Problem: old requests delay new ones

3. FIXED WINDOW COUNTER
   • Simple: count resets every window
   • Problem: boundary spike (2x traffic at window edge)

4. SLIDING WINDOW LOG
   • Store timestamp of each request
   • Count requests in [now-window, now]
   • Accurate but memory expensive

5. SLIDING WINDOW COUNTER ■ BEST BALANCE
   • current_window_count + prev_window_count * overlap_ratio
   • Formula: count = curr + prev * (window - elapsed)/window

REDIS IMPLEMENTATION:
  INCR user:{id}:window:{timestamp}
  EXPIRE user:{id}:window:{timestamp} {window_size}
  → Atomic via Lua script or MULTI/EXEC
```

F. DATABASE & API DESIGN

REDIS SCHEMA:

```
  Key: rate:{user_id}:{window_start_ts}
  Value: integer count
  TTL: window size

RULES CONFIG (DB/YAML):
  {
    "endpoint": "/api/tweet",
    "limit": 300,
    "window": 10800,  // 3 hours
    "key_by": "user_id"
  }

RESPONSE HEADERS:
  X-RateLimit-Limit: 100
  X-RateLimit-Remaining: 42
  X-RateLimit-Reset: 1686000000
  Retry-After: 60  (on 429)
```

G. SCALING & BOTTLENECKS

CHALLENGE: Distributed rate limiting

```
  → Naive: each server has local counter → inaccurate
  → Solution 1: Centralized Redis (single source of truth)
  → Solution 2: Sticky sessions (IP hash LB)
  → Solution 3: Eventual consistency (slight over-allow)

REDIS RACE CONDITION:
  Problem: INCR not atomic with GET+conditional
  Fix: Lua script (atomic execution)
  Fix: Redis MULTI/EXEC (optimistic locking)

FAILURE MODES:
  • Redis down → fail open (allow all) or fail closed (block all)
  • ■ "Choose fail-open for availability, fail-closed for security"
  • Use circuit breaker pattern

SCALING REDIS:
  • Redis Cluster: hash slot sharding
  • Read replicas for rule lookups
  • Local cache for rules (TTL 60s)
```

H. ADVANCED INSIGHTS

MULTI-TIER RATE LIMITING:

```
  Layer 1: IP-level (prevents DDoS)
  Layer 2: User-level (prevents abuse)
  Layer 3: API-key level (monetization)
  Layer 4: Endpoint-level (protect expensive ops)

EDGE CASES:
  • Clock skew across servers → use NTP / Redis server time
  • User with multiple IPs → user_id as primary key
  • Shared IP (NAT/proxy) → combo of IP + user_id

MONITORING:
  • Rate limit hit ratio per endpoint
  • P99 latency of rate limit check
  • Redis memory usage
  • Alert on >5% requests being throttled

COST:
  • Redis ~1KB per user per window
  • 10M users = 10GB Redis RAM → manageable
```

I. TRADE-OFFS

Token Bucket vs Sliding Window:

```
  + Token bucket: allows burst, simple
  - Token bucket: hard to implement distributed

Centralized vs Distributed counter:
  + Centralized: accurate
  - Centralized: single point of failure, latency

Fail-open vs Fail-closed:
  + Fail-open: better availability
  - Fail-open: security risk during Redis outage

■ MUST SAY: "I'd use sliding window counter with Redis Lua
scripts for atomicity, fail-open with circuit breaker,
and multi-tier rules: IP → User → API Key"
```

J. INTERVIEW TIPS

✓ Always ask: "Should we rate limit by IP, user, or API key?"

✓ Mention distributed challenge immediately

✓ Know all 5 algorithms + trade-offs

✓ Talk about headers (X-RateLimit-*)

✓ Address Redis failure scenario

✗ Don't forget: rules must be configurable (not hardcoded)

✗ Don't ignore the "what if Redis is down?" question

> `MNEMONIC: "TLFSC" → Token, Leaky, Fixed, Sliding-log, Sliding-counter`

2 Consistent Hashing

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Problem: With N servers, key goes to server = hash(key) % N

```
• Adding/removing server → rehash ALL keys → cache misses, thundering herd
• Solution: Consistent hashing → only K/N keys remapped on change
• Used in: Cassandra, DynamoDB, Chord DHT, CDNs, memcached
```

B. HOW IT WORKS

HASH RING (0 to 2^32-1):

0

```
   ■■■■■■■■
 /  S1      ■    ●     ■  ← Server mapped to ring position
■  K2●      ■  ← Key mapped to ring position
■  S2●      ■  → Key assigned to NEXT server clockwise
■    K1●    ■
 \  S3    /
  ■■■■■■■■
 2^32-1

ASSIGNMENT RULE:
  Walk clockwise from key's position → first server = owner

NODE ADD (S4 between S1 and S2):
  → Only keys between S1 and S4 reassigned to S4
  → ~K/N keys affected (vs ALL in naive hashing)

NODE REMOVE:
  → Keys from removed node go to next clockwise server
```

C. VIRTUAL NODES (VNODES) ■

PROBLEM with basic consistent hashing:

```
  • Non-uniform distribution (hot spots)
  • When node removed, all load goes to ONE next node

SOLUTION: Virtual Nodes
  • Each physical server = 100-200 virtual nodes on ring
  • S1 → [S1_v1, S1_v2, ..., S1_v150] scattered on ring
  • Load distributes evenly across all physical nodes

FORMULA: num_vnodes = ring_size / expected_nodes * load_factor

HETEROGENEOUS CLUSTERS:
  • Powerful server → more vnodes → more load
  • S1 (32GB RAM) → 200 vnodes
  • S2 (8GB RAM) → 50 vnodes

TRADE-OFF:
  + Even distribution, handles hotspots
  - More memory for vnode-to-server mapping
  - Rebalancing more complex
```

D. REPLICATION

CASSANDRA PATTERN:

• Replication factor = 3

```
    • Key → first 3 DISTINCT physical servers clockwise
    • Coordinator node routes to replicas

  RING VIEW:
    Key K → S1 (primary), S2 (replica-1), S3 (replica-2)

  CONSISTENCY LEVELS:
    • ONE: ack from 1 replica (fast, less consistent)
    • QUORUM: ack from (N/2)+1 replicas ← most common
    • ALL: ack from all replicas (slow, fully consistent)
```

E. PARTITIONING STRATEGIES

CONSISTENT HASHING vs RANGE-BASED:

```
    Consistent Hashing:
      + Even distribution
      + Simple add/remove nodes
      - No range queries
      Use: Cache, KV store (DynamoDB, Cassandra)

    Range-Based (HBase, Bigtable):
      + Range queries possible
      - Risk of hotspots (sequential keys)
      Use: Time-series data, sorted data

  RENDEZVOUS HASHING (alternative):
    • For each key, compute hash(key, server) for ALL servers
    • Key goes to server with HIGHEST hash value
    • No ring needed, simpler but O(N) lookup
```

F. REAL-WORLD APPLICATIONS

DYNAMO/CASSANDRA:

• Consistent hashing + vnodes

• Gossip protocol for membership

• Merkle trees for anti-entropy

CDN (Akamai, CloudFront):

• Route requests to nearest edge server

• Consistent hashing for cache routing

LOAD BALANCERS:

• Sticky sessions via consistent hashing on IP

• Upstream server selection

REDIS CLUSTER:

• 16384 hash slots

• Each node owns a range of slots

• Similar to consistent hashing concept

G. INTERVIEW TIPS

✓ Draw the ring diagram

✓ Explain virtual nodes unprompted

✓ Mention: only K/N keys remapped (vs all in mod-N)

✓ Connect to real systems: Cassandra, DynamoDB

✓ Discuss replication with ring traversal

■ MUST SAY IN INTERVIEW:

"Virtual nodes solve both the uneven distribution problem and the hot-spot problem when nodes fail"

COMMON MISTAKES:

```
  ✗ Forgetting virtual nodes → interviewer will probe this
  ✗ Not explaining WHY consistent hashing > mod-N
  ✗ Ignoring replication factor discussion

MNEMONIC: "RING-V" → Ring, Incremental rehash, No hotspots,
  Goes clockwise, Virtual nodes
```

3 Key-Value Store

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Distributed KV store: get(key), put(key, value), delete(key)

• Examples: DynamoDB, Cassandra, Redis, RocksDB

• Use cases: User sessions, shopping carts, config, cache

```
• Challenge: CAP theorem → pick 2 of 3 (Consistency, Availability, Partition-tolerance)
```

B. REQUIREMENTS

FUNCTIONAL:

```
  • put(key, value)  → store or update
  • get(key) → retrieve
  • delete(key) → remove
  • Keys: up to 10KB, Values: up to 10MB

NON-FUNCTIONAL (scale matters!):
  • 10M DAU, 10:1 read:write ratio
  • P99 read latency <10ms
  • 99.99% availability
  • 10PB total data
  • Auto-scaling
  • Tunable consistency
```

C. HIGH-LEVEL DESIGN

Client

```
    ↓
  [Coordinator Node] ← any node can be coordinator
    ↓
  [Consistent Hash Ring] → determine which nodes own key
    ↓
  [N replica nodes] ← replication factor = 3

  ARCHITECTURE:
  ■■■■■■■■■   ■■■■■■■■■   ■■■■■■■■■
  ■ Node1■■■■■■ Node2■■■■■■ Node3■
  ■ (P)  ■    ■ (R1) ■    ■ (R2) ■
  ■■■■■■■■■   ■■■■■■■■■   ■■■■■■■■■
      ↑ gossip protocol (membership)

  Each node has:
    • Consistent hash ring knowledge
    • Local storage engine (LSM tree / RocksDB)
    • Bloom filter (quick miss detection)
    • Merkle tree (anti-entropy sync)
```

D. STORAGE ENGINE (DEEP DIVE)

MEMORY: Hash table for hot data + WAL for durability

LSM TREE (Log-Structured Merge Tree) ■:

```
  Write path:
    1. Write to WAL (crash recovery)
    2. Write to MemTable (sorted in-memory)
    3. MemTable full → flush to SSTable on disk
    4. Background: compact SSTables

  Read path:
    1. Check MemTable (fastest)
    2. Check Bloom filter (skip SSTable if key doesn't exist)
    3. Search SSTables newest → oldest

  WHY LSM > B-Tree for writes:
    LSM: sequential disk writes (fast)
    B-Tree: random disk writes (slow)
    Trade-off: LSM has write amplification during compaction

BLOOM FILTER:
  • Probabilistic: "key definitely NOT present" or "maybe present"
  • False positives OK (just do disk read), false negatives NOT OK
  • Size: ~10 bits per key → 1% false positive rate
```

E. CONSISTENCY & REPLICATION

DYNAMO-STYLE (N, W, R):

N = total replicas

```
  W = write quorum (must ack)
  R = read quorum (must return)

  Strong consistency: W + R > N  (e.g., N=3, W=2, R=2)
  Eventual consistency: W=1, R=1 (fast, may be stale)
  ■ "Tune W+R vs N based on use case"

VERSIONING (handle conflicts):
  • Vector clocks: [S1:3, S2:1] per value version
  • On conflict → return both versions to client (let client resolve)
  • Last-Write-Wins (LWW): use timestamp (risk: data loss)

ANTI-ENTROPY (background sync):
  • Merkle trees: compare tree hashes to find divergent data
  • Hinted handoff: if node down, neighbor stores hint, delivers when back
  • Read repair: fix stale data during reads
```

F. FAILURE HANDLING

FAILURE DETECTION: Gossip Protocol

• Each node periodically pings random nodes

```
  • Propagates node state (alive/suspected/dead)
  • O(log N) propagation time

SLOPPY QUORUM:
  • If target node down → use next healthy node ("sloppy quorum")
  • Data stored with hint for failed node
  • When failed node recovers → hinted handoff transfers data

SPLIT BRAIN:
  • Network partition → 2 groups think other is dead
  • Both continue writes → conflicts
  • Resolution: vector clocks + merge on reconnect

PERMANENT FAILURE:
  • Merkle tree comparison per vnodes
  • Sync only divergent subtrees (efficient)
```

G. TRADE-OFFS (CAP)

CP SYSTEMS (Consistency + Partition tolerance):

• HBase, Zookeeper

• Reject writes if can't reach quorum

```
    • Good for: financial data, inventory

AP SYSTEMS (Availability + Partition tolerance):
  • DynamoDB, Cassandra, CouchDB
  • Return possibly stale data
  • Good for: shopping carts, social feeds

■ "DynamoDB/Cassandra = AP by default, tunable toward CP"
■ "No system achieves all 3 of CAP simultaneously"

PACELC (extension of CAP):
  Even without partition:
  Trade latency vs consistency
  DynamoDB: PA/EL (available on partition, low latency else)
```

H. INTERVIEW TIPS

✓ Start with CAP theorem, explain trade-offs

✓ Discuss LSM tree vs B-tree for storage

✓ Mention bloom filters for read optimization

✓ Explain vector clocks for conflict resolution

✓ Cover gossip protocol for failure detection

✓ Tie (N,W,R) to consistency-availability trade-off

■ MUST SAY IN INTERVIEW:

"I'd use LSM tree + bloom filter for storage,
consistent hashing + vnodes for partitioning,
gossip for failure detection, vector clocks for versioning"

```
MNEMONIC: "CLBGV" → Consistent-hash, LSM, Bloom-filter,
  Gossip, Vector-clocks
```

4 Unique ID Generator

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Generate globally unique IDs across distributed systems

• Examples: Twitter Snowflake, Instagram IDs, UUID

• Use cases: DB primary keys, distributed tracing, order IDs

• Constraints: unique, sortable by time, 64-bit, 10K IDs/sec per machine

B. APPROACHES

1. UUID (128-bit)

+ Simple, no coordination needed

- Not sortable, too large, not readable

```
   Use: when uniqueness >> sortability

2. DB AUTO_INCREMENT
   + Simple, sortable
   - Single point of failure, bottleneck at scale

3. TICKET SERVER (Flickr approach)
   • Centralized auto-increment DB server
   + Sortable, numeric
   - Single point of failure

4. TWITTER SNOWFLAKE ← ■ BEST ANSWER
   • 64-bit ID = timestamp(41) + datacenter(5) + machine(5) + seq(12)
```

C. SNOWFLAKE DEEP DIVE ■

BIT LAYOUT (64 bits total):

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 1  ■       41         ■    5     ■    5     ■    12      ■
■sign■   timestamp(ms)  ■datacenter■ machine  ■  sequence  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

sign: always 0 (positive)
timestamp: ms since custom epoch (2010-11-04)
  → 41 bits = 69 years of timestamps
datacenter_id: 5 bits = 32 datacenters
machine_id: 5 bits = 32 machines per DC
sequence: 12 bits = 4096 IDs per ms per machine
  → Total: 32 × 32 × 4096 = 4M IDs/ms globally

CAPACITY:
  • 41 bits → ~69 years (until ~2079)
  • 4096 seq IDs/ms per machine
  • 1024 machines × 4096 = 4M IDs/ms total

SORTABLE: Yes! Higher timestamp = higher ID value

CODE LOGIC:
  ts = current_ms - custom_epoch
  if ts == last_ts:
    seq = (seq + 1) & 0xFFF  // max 4095
    if seq == 0: wait_next_ms()
  else:
    seq = 0
  id = (ts << 22) | (dc_id << 17) | (machine_id << 12) | seq
```

D. CLOCK SKEW PROBLEM

PROBLEM:

• NTP can adjust clocks backward

```
  • Machine A time > Machine B time → IDs from B may be "old"
  • Can cause duplicate IDs or non-monotonic ordering

SOLUTIONS:
  1. Refuse to generate IDs if clock moves backward
     → Wait until clock catches up
  2. Use last known time + sequence extension
  3. Borrow bits from sequence for extra monotonicity
  4. Use logical clocks (Lamport timestamps)

■ "Clock skew is the #1 challenge in distributed ID generation"
```

E. ALTERNATIVES

ULID (Universally Unique Lexicographically Sortable ID):

```
  • 128 bits: 48-bit timestamp + 80-bit random
  • URL-safe base32 encoding
  • Monotonically sortable

KSUID (K-Sortable Unique ID):
  • 160 bits: 32-bit timestamp + 128-bit random
  • Slightly larger but very low collision probability

INSTAGRAM'S APPROACH:
  • 64-bit: timestamp(41) + shard_id(13) + auto_increment(10)
  • Shard ID from Postgres logical shards

MONGODB OBJECTID:
  • 96 bits: timestamp(32) + machine(24) + pid(16) + counter(24)
```

F. TRADE-OFFS

Snowflake:

+ Fast, sortable, readable, 64-bit

- Requires synchronized clocks, machine ID coordination

UUID:

+ Simple, no coordination

```
  - Not sortable, large (128-bit), random distribution = poor index performance

DB sequence:
  + Simple, perfect ordering
  - Bottleneck, SPOF

■ "For most distributed systems: Snowflake is the answer.
  For simplicity with eventual sortability: ULID"
```

G. INTERVIEW TIPS

✓ Immediately propose Snowflake structure

✓ Draw the bit layout diagram

✓ Explain capacity calculation (how many IDs/sec)

✓ Address clock skew proactively

✓ Know the epoch customization trick

COMMON MISTAKES:

✗ Suggesting UUID without knowing its trade-offs

✗ Not discussing clock skew

✗ Forgetting that Snowflake needs machine ID assignment

```
MNEMONIC: "TDMS" → Timestamp, Datacenter, Machine, Sequence
```

5 URL Shortener

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

```
• Convert long URL → short URL (e.g., bit.ly/abc123)
• Redirect short URL → original URL
• Real-world: bit.ly, TinyURL, t.co (Twitter)
• Core challenge: generate unique short codes at scale
```

B. REQUIREMENTS

FUNCTIONAL:

• POST /shorten {longUrl, [customAlias], [expiry]}

```
  • GET /{shortCode} → 301/302 redirect
  • DELETE /{shortCode}
  • Analytics: click count, geo, referrer (optional)

NON-FUNCTIONAL:
  • 100M URLs shortened/day = ~1200 write/sec
  • 10:1 read:write → 10B redirects/day = 120K read/sec
  • Low latency redirect (<10ms)
  • URLs don't change (permanent shortening)
  • 10-year data retention = 365B URLs

ESTIMATION:
  • 100M/day writes, 365 days × 10 years = 365B records
  • Avg URL = 100 bytes → 36.5TB storage
  • Short code: 7 chars (base62) → 62^7 = 3.5 trillion combos
```

C. SHORT CODE GENERATION

OPTION 1: HASH + TRUNCATE

```
    md5("https://long-url.com") → 128-bit hash
    Take first 7 chars of base62 encoding
    Problem: Collisions!
    Fix: If collision, append counter and rehash

  OPTION 2: SNOWFLAKE ID → BASE62 ■ BEST
    Generate unique 64-bit ID (Snowflake)
    Convert to base62: [0-9a-zA-Z]
    62^7 = 3.5 trillion → enough for any scale
    NO collision possible (IDs are unique)
    Sortable by creation time!

  BASE62 ENCODING:
    Alphabet: 0-9, a-z, A-Z (62 chars)
    7 chars → 62^7 ≈ 3.5 trillion unique codes
    Example: 12345678 → "8M0kX"

  OPTION 3: RANDOM STRING
    Generate random 7-char base62 string
    Check DB for collision, regenerate if exists
    Problem: DB check on every generation
```

D. HIGH-LEVEL DESIGN

WRITE PATH:

```
  Client → API Server → ID Generator
             ↓
          [Check if custom alias exists]
             ↓
          [Store: short_code → long_url]
             ↓ (DB + Cache)
          Return short URL

READ PATH (Critical - must be FAST):
  Client → [CDN / Cache] → API Server
                              ↓
                          [Redis Cache]
                              ↓ (miss)
                          [Database]
                              ↓
                          301/302 Redirect

301 vs 302:
  301 Permanent: browser caches, server gets no analytics
  302 Temporary: browser always hits server ← better for analytics
  ■ "Use 302 if you need click analytics"
```

E. DATABASE DESIGN

TABLE: urls

```
  short_code  VARCHAR(7)  PRIMARY KEY
  long_url    TEXT        NOT NULL
  user_id     BIGINT      FK
  created_at  TIMESTAMP
  expires_at  TIMESTAMP   NULLABLE
  click_count BIGINT      DEFAULT 0

TABLE: users
  user_id     BIGINT      PRIMARY KEY
  email       VARCHAR
  api_key     VARCHAR

WHICH DB?
  → NoSQL (DynamoDB/Cassandra) preferred:
    • Simple key-value lookups
    • Easy horizontal scaling
    • short_code as partition key
  → SQL if you need complex analytics

CACHING:
  • Redis: short_code → long_url
  • LRU eviction (hot URLs stay cached)
  • TTL = expiry of URL
  • Cache hit rate should be >90% (hot URLs reused often)
```

F. SCALING

READ SCALING:

```
  • CDN for most popular URLs (cache at edge)
  • Redis cluster for hot URLs
  • Read replicas for DB

WRITE SCALING:
  • Horizontal API servers (stateless)
  • Snowflake ID generator per machine
  • Database sharding on short_code hash

ANALYTICS (separate pipeline):
  Client click → API → Kafka → Stream Processor
                                      ↓
                              Analytics DB (Clickhouse/Redshift)

CLEANUP:
  • Background job to delete expired URLs
  • Soft delete first, hard delete after 30 days (reclaim storage)
  • Recycle short codes after expiry (careful: cached URLs!)
```

G. TRADE-OFFS & EDGE CASES

Custom aliases:

• Allow custom = need uniqueness check

```
  • Charge premium for vanity URLs (business)

Malicious URLs:
  • Check against Google Safe Browsing API
  • Rate limit per user/IP

Same long URL:
  • Return existing short code? Or create new one?
  • "Idempotent shortening" = same input → same output
  • Trade-off: requires lookup before insert

Hot keys in cache:
  • Viral URL → thundering herd on cache miss
  • Solution: Cache warming, probabilistic early expiration

■ MUST SAY: "I'd use Snowflake IDs → base62, NoSQL for storage,
  Redis for caching, 302 redirects for analytics"
```

H. INTERVIEW TIPS

✓ Calculate: 62^7 ≈ 3.5 trillion (justify 7 chars)

✓ Discuss 301 vs 302 redirect trade-off

✓ Address hash collision handling

✓ Mention cache-aside pattern with Redis

✓ Separate analytics pipeline (don't block redirect)

```
MNEMONIC: "SHAPE" → Snowflake, Hash→base62, Analytics-async,
  Postgres/NoSQL, Edge-cache
```

6 Web Crawler

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Automatically browses web to download/index content
• Used by: Google (Googlebot), Bing, SEO tools, archival (Wayback Machine)
• Challenges: scale (billions of pages), politeness, duplicate detection
• Must handle: redirects, robots.txt, dynamic JS pages, traps

B. REQUIREMENTS

FUNCTIONAL:

• Start from seed URLs, discover new URLs
• Download HTML content
• Store crawled content
• Re-crawl periodically (freshness)
• Respect robots.txt and crawl rate

NON-FUNCTIONAL:

• Scale: 1B pages, ~10TB/day download
• Politeness: max 1 req/sec per domain
• Distributed: 1000s of workers
• Fault tolerant (resume on failure)
• Extensible (PDFs, images, not just HTML)

ESTIMATION:

• 1B pages × 500KB avg = 500TB total
• 1M pages/day × 500KB = 500GB/day
• 1M pages/day = ~12 pages/sec

C. HIGH-LEVEL DESIGN

D. DEEP DIVE COMPONENTS

URL FRONTIER (most important component):

```
  Two-queue architecture:
    1. Priority Queue: by importance score
       (PageRank, freshness, crawl frequency)
    2. Politeness Queue: per-domain FIFO
       (ensure max 1 req/sec per domain)

  Priority Mapping:
    URL → priority score → priority bucket
    Priority queue → domain → politeness queue
    Fetcher picks from politeness queue

DUPLICATE DETECTION:
  Content-level: Hash(content) → seen-hashes set
  URL-level: Bloom filter (URL seen before?)
    → False positive = skip URL (OK, small loss)
    → False negative impossible → no duplicates slip through
  Near-duplicate: SimHash / MinHash (similar content)

ROBOTS.TXT:
  Cache per domain (re-fetch weekly)
  Parse: Disallow, Allow, Crawl-delay directives
  Always respect (legal + ethical)

LINK EXTRACTION:
  Parse HTML → find <a href>, <link>, <script src>
  Normalize URLs: resolve relative, lowercase, remove fragments
  Filter: same-domain vs cross-domain seeds
```

E. SCALING & FAULT TOLERANCE

DISTRIBUTED CRAWLING:

• Partition URL space by domain hash across worker nodes

• Each worker responsible for set of domains

• Consistent hashing for worker assignment

STORAGE:

```
  • Raw HTML → S3/HDFS (cheap, durable)
  • URL metadata → Cassandra (distributed KV)
  • URL frontier → Redis sorted sets (priority + delay)

FAULT TOLERANCE:
  • Checkpointing: periodically save frontier state
  • Exactly-once via URL dedup + idempotent storage
  • Dead letter queue for failed URLs (retry with backoff)

DYNAMIC PAGES (JS-rendered):
  • Use headless Chrome (Puppeteer)
  • Expensive: only for important domains
  • Render → extract HTML → parse

SPIDER TRAPS:
  • Infinite URL generation (/date/2024/01/01, /date/2024/01/02...)
  • Fix: max URL depth limit (e.g., 10 hops)
  • Fix: max URLs per domain
  • Fix: detect calendar/repetitive patterns
```

F. TRADE-OFFS

BFS vs DFS:

```
  BFS preferred → discovers high-priority pages first
  DFS → prone to getting stuck in traps

Politeness vs Speed:
  Too fast → IP banned, hostile
  Too slow → low coverage

Recrawl frequency:
  News sites: hourly
  Static pages: monthly
  Formula: freshness = f(change_rate, importance)

■ "The URL Frontier is the heart of the crawler.
  Get the priority + politeness balance right."
```

G. INTERVIEW TIPS

✓ Propose two-queue frontier (priority + politeness)

✓ Mention bloom filter for URL deduplication

✓ Address robots.txt compliance

✓ Discuss spider trap detection

✓ Mention content dedup via hashing / SimHash

COMMON MISTAKES:

✗ Designing single-threaded crawler

```
  ✗ Not mentioning politeness (domain rate limiting)
  ✗ Ignoring duplicate content detection

MNEMONIC: "FDPRS" → Frontier, Dedup, Politeness, Robots, SimHash
```

7 Notification System

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

- Send notifications across channels: push, email, SMS
- Examples: Facebook alerts, Uber ride updates, bank OTPs
- Challenges: scale, reliability, provider failures, user preferences
- Types: Push (iOS/Android), Email, SMS, In-app

B. REQUIREMENTS

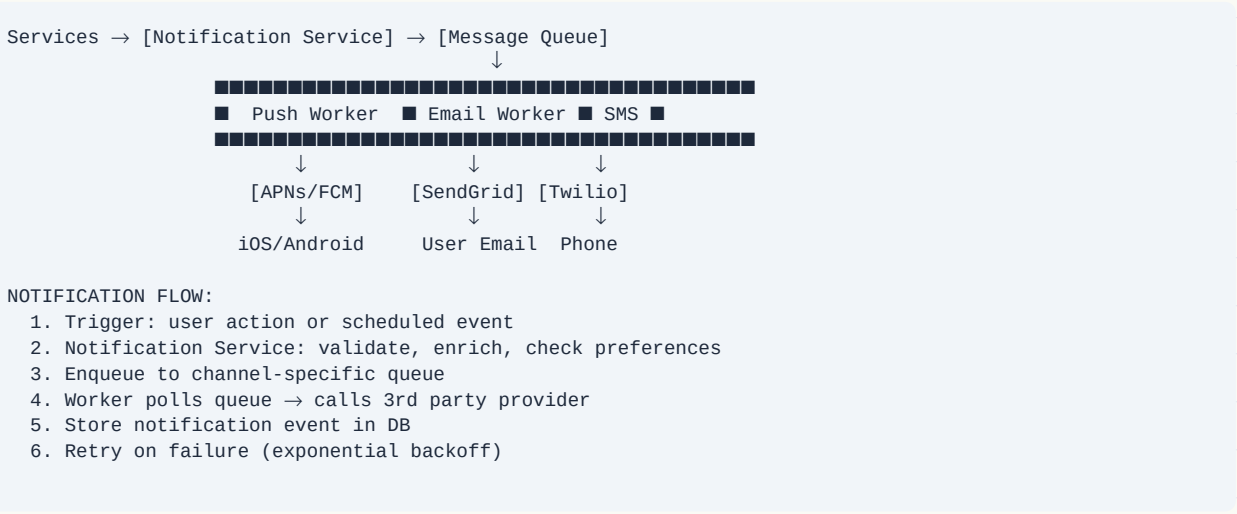
FUNCTIONAL:

- Send push/email/SMS notifications
- Support scheduling and retry
- User opt-out / preferences
- Template management
- Notification history

NON-FUNCTIONAL:

- 10M mobile push/day
- 1M SMS/day
- 5M emails/day
- Soft real-time (<1 min for time-sensitive)
- At-least-once delivery
- High availability (>99.9%)

C. HIGH-LEVEL DESIGN




```
NOTIFICATION FLOW:
  1. Trigger: user action or scheduled event
  2. Notification Service: validate, enrich, check preferences
  3. Enqueue to channel-specific queue
  4. Worker polls queue → calls 3rd party provider
  5. Store notification event in DB
  6. Retry on failure (exponential backoff)
```

D. PUSH NOTIFICATIONS

iOS (APNs - Apple Push Notification Service):

```
  • Device token required (register at app launch)
  • JWT auth between server and APNs
  • Payload: max 4KB
  • Quality of Service: store-and-forward for offline devices

Android (FCM - Firebase Cloud Messaging):
  • Registration token (similar to APNs)
  • Supports topics (broadcast to group)
  • Payload: max 4KB data, unlimited notification

FLOW:
  App → APNs/FCM (get device token)
  App → Our server (register device token)
  Our server → APNs/FCM → Device

TOKEN MANAGEMENT:
  • Tokens expire/change (app reinstall, OS update)
  • Handle feedback service: remove stale tokens
  • Store: user_id → [device_token1, token2, ...]
```

E. DATABASE DESIGN

TABLE: notifications

notif_id UUID PK

user_id BIGINT FK

```
  channel     ENUM        (push/email/sms)
  status      ENUM        (pending/sent/failed/delivered)
  content     JSONB
  created_at  TIMESTAMP
  sent_at     TIMESTAMP
  retry_count INT         DEFAULT 0

TABLE: user_preferences
  user_id     BIGINT      PK
  channel     ENUM
  enabled     BOOLEAN
  quiet_hours JSONB       {start: 22, end: 8, tz: "UTC-5"}

TABLE: device_tokens
  user_id     BIGINT
  token       VARCHAR     UNIQUE
  platform    ENUM        (ios/android/web)
  updated_at  TIMESTAMP

TEMPLATES (stored separately):
  template_id, channel, content_template, variables
```

F. RELIABILITY & SCALING

MESSAGE QUEUE (Kafka/SQS):

```
  • Per-channel topics: push_notif, email_notif, sms_notif
  • Consumer groups per provider
  • Dead letter queue for persistent failures

RETRY STRATEGY:
  • Exponential backoff: 1s, 2s, 4s, 8s...
  • Max retries: 3-5
  • Failed → DLQ → alert on-call

RATE LIMITING (per provider):
  • APNs: no stated limit but recommend <1000/sec/connection
  • Twilio: depends on plan
  • SendGrid: 600K emails/hour on enterprise

DEDUPLICATION:
  • idempotency_key = hash(user_id + template_id + ref_id + window)
  • Store in Redis with TTL = dedup window
  • Prevent duplicate sends from retries

ANALYTICS:
  • Track: sent, delivered, opened, clicked
  • Provider webhooks → Kafka → analytics DB
```

G. EDGE CASES

Notification storms:

• Marketing blast to 100M users

```
  • Solution: rate limiting + throttling at queue level
  • Spread over time window

Provider outage:
  • Primary APNs down → circuit breaker → failover to backup
  • SMS: Twilio → Vonage failover

Quiet hours / timezone:
  • Store user timezone in preferences
  • Delay non-urgent notifications until morning

Do-Not-Disturb (DND):
  • Respect OS-level DND on iOS
  • Allow override for critical notifications

■ "At-least-once delivery is easier than exactly-once.
  Use idempotency keys to prevent user-visible duplicates"
```

H. INTERVIEW TIPS

✓ Cover all 4 channels: push (iOS+Android), email, SMS

✓ Draw the queue-based architecture

✓ Discuss provider failures + circuit breaker

✓ Mention user preferences + quiet hours

✓ Address deduplication via idempotency key

■ MUST SAY IN INTERVIEW:

"Use message queues to decouple notification generation from delivery, with per-channel workers"

```
MNEMONIC: "PQRDT" → Providers, Queues, Retry, Dedup, Tracking
```

8 News Feed System

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Show personalized feed of posts from followed users

• Examples: Facebook News Feed, Twitter Timeline, Instagram Feed

```
• Core challenge: fanout (1 post → millions of followers)
• Two models: Fan-out on write (push) vs Fan-out on read (pull)
```

B. REQUIREMENTS

FUNCTIONAL:

• User publishes post

• User sees feed from followed accounts

• Feed sorted by relevance/time

• Support text, images, video

NON-FUNCTIONAL:

• 10M DAU, avg 500 friends/user

• Feed load: <1 sec

• Post publish: <5 sec

• High availability (AP system)

• Eventual consistency OK for feed

C. HIGH-LEVEL DESIGN

POST FLOW:

D. FANOUT STRATEGIES (CRITICAL)

PUSH (fan-out on write):

```
  Post created → immediately push to all followers' feeds
  Pros:
    + Feed fetch is O(1) from cache
    + Low read latency
  Cons:
    - Celebrity problem: 10M followers → 10M writes
    - Hot users → write amplification
    - Wastes computation for inactive users

PULL (fan-out on read):
  Feed fetch → pull recent posts from all followees
  Pros:
    + No wasted work for inactive users
    + Handles celebrities simply
  Cons:
    - Slow for users with many followees
    - DB hit every feed load

■ HYBRID APPROACH (Facebook/Twitter strategy):
  • Regular users (< N followers): fan-out on WRITE
  • Celebrity users (> N followers): fan-out on READ
  • Mix at read time: cached feed + real-time celebrity posts

THRESHOLD: N ≈ 10,000 followers (tune based on system)
```

E. FEED RANKING

SIMPLE: Reverse chronological (Twitter classic)

ML RANKING (Facebook approach):

```
  Features:
    • Affinity score (how often you interact with poster)
    • Post type weight (video > photo > text)
    • Time decay (recent > old)
    • Engagement score (likes, comments)

  Edge Rank (Facebook simplified):
    Score = affinity × weight × time_decay

SERVING:
  Pre-computed feed in Redis:
    user:{id}:feed → sorted set of post_ids (score = timestamp)
  Fetch post details from Post Cache (separate)
  Paginate: ZREVRANGE user:123:feed 0 19 (top 20)

PAGINATION:
  Cursor-based: last_seen_post_id (not page number)
  Why: items added to feed between page loads
```

F. DATABASE DESIGN

TABLE: posts

```
  post_id   BIGINT (Snowflake)  PK
  user_id   BIGINT              FK
  content   TEXT
  media_url VARCHAR
  created_at TIMESTAMP

TABLE: follows
  follower_id  BIGINT
  followee_id  BIGINT
  PRIMARY KEY (follower_id, followee_id)
  INDEX on followee_id (to find all followers of X)

REDIS SCHEMA:
  feed:{user_id} → Sorted Set (score=timestamp, value=post_id)
  post:{post_id} → Hash (user_id, content, likes, ...)
  user:{user_id} → Hash (name, avatar, ...)

GRAPH DB for follows? Only if complex social queries needed.
```

G. SCALING

BOTTLENECKS:

```
  1. Celebrity writes → fan-out on read for them
  2. Feed cache eviction → keep only last 200-500 posts in cache
  3. Graph queries (get followers) → graph DB or inverted index

MULTI-REGION:
  • Replicate feed cache per region
  • User's feed served from nearest region
  • Post published → async propagation to all regions

MEDIA:
  • Images/videos → CDN (CloudFront, Akamai)
  • Post only stores URL, not raw media

■ "Fan-out on write for regular users, fan-out on read
  for celebrities. Merge at read time."
```

H. INTERVIEW TIPS

✓ Immediately discuss push vs pull vs hybrid

✓ Explain celebrity/hotspot problem

✓ Propose Redis sorted set for feed storage

✓ Mention cursor-based pagination

✓ Discuss eventual consistency (OK for feed)

COMMON MISTAKES:

✗ Pure push model without addressing celebrity problem

✗ Not discussing feed ranking

```
  ✗ Page-number pagination (breaks with real-time updates)

MNEMONIC: "PUSH-PULL-HYBRID → Regular-Celebrity-Mix"
```

9 Chat System

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Real-time messaging between users (1:1 and group)
• Examples: WhatsApp, Slack, Facebook Messenger
• Key challenge: Real-time bi-directional communication + offline handling
• Types: 1-on-1, group chat, channels

B. REQUIREMENTS

FUNCTIONAL:

• Send/receive messages in real-time
• 1:1 and group chat (max 500 members)
• Online presence (typing indicator, last seen)
• Message history (persistent)
• Push notification when offline
• Read receipts (sent/delivered/read)

NON-FUNCTIONAL:

• 50M DAU, avg 40 messages/day = 2B messages/day
• <100ms delivery latency
• High availability (AP system)
• Messages persist forever
• E2E encryption (optional, mention it)

C. REAL-TIME COMMUNICATION

OPTIONS:

```
1. HTTP Polling: client polls every N seconds
   - Wastes bandwidth, high latency
2. Long Polling: server holds request until message
   - Better but still half-duplex, connection issues
3. WebSocket ■ BEST:
   - Full-duplex, persistent TCP connection
   - Low overhead after handshake
   - Server can push anytime
4. SSE (Server-Sent Events): server→client only
   - Good for notifications, not chat

WEBSOCKET FLOW:
  Client ←—— HTTP Upgrade ——→ Chat Server
  Client ←■■ WebSocket (TCP) ■■→ Chat Server
  (persistent, bidirectional, low latency)
```

D. HIGH-LEVEL DESIGN


```
KEY INSIGHT: Chat servers are stateful (WS connections)
  → Can't use simple LB (must route to correct server)
  → Use service registry (ZooKeeper) to find user's server
```

E. MESSAGE DELIVERY

USER ONLINE (same server): direct via WebSocket

USER ONLINE (different server):

```
  Msg → Kafka → subscriber on User B's server → WS push

USER OFFLINE:
  Msg stored in DB
  Push notification sent (APNs/FCM)
  When user comes online: sync unread messages

SEQUENCE IDs:
  Problem: message ordering across servers
  Solution: Snowflake IDs per message (time-ordered)
  Client sorts by message_id

ACK PROTOCOL (WhatsApp-style):
  1 tick: sent to server
  2 ticks: delivered to recipient device
  2 blue ticks: read by recipient
  → Server tracks delivery status per message
```

F. DATABASE DESIGN

WHY CASSANDRA? ■

```
  • Write-heavy (2B messages/day)
  • Time-series queries (messages in a conversation)
  • Horizontally scalable
  • High write throughput
  • Tunable consistency

TABLE: messages
  channel_id  UUID   PARTITION KEY
  message_id  BIGINT CLUSTERING KEY (Snowflake, desc)
  sender_id   UUID
  content     TEXT
  type        ENUM (text/image/video)
  created_at  TIMESTAMP
  PRIMARY KEY (channel_id, message_id)
  → Efficiently fetch: last N messages in channel

TABLE: channel_members
  channel_id  UUID   PARTITION KEY
  user_id     UUID   CLUSTERING KEY
  joined_at   TIMESTAMP

TABLE: user_channels
  user_id     UUID   PARTITION KEY
  channel_id  UUID
  last_read_id BIGINT
```

G. ONLINE PRESENCE

HEARTBEAT MECHANISM:

Client sends heartbeat every 5 seconds

Server marks user online/offline in Redis

```
  Key: presence:{user_id} → TTL=10s (expire if no heartbeat)

SUBSCRIPTION MODEL:
  User A wants to see User B's status
  User A's server subscribes to B's presence channel (Pub/Sub)
  When B's status changes → publish → A's server → A's WS

STATUS TYPES:
  • Online (heartbeat active)
  • Away (no heartbeat > 5min but session open)
  • Offline (session closed)
  • Do Not Disturb (explicit user setting)

SCALE:
  50M online users × 500 friends = 25B subscriptions
  → Too many pub/sub channels
  → Solution: only subscribe when user opens conversation
  → Or: batch presence updates every 30s
```

H. TRADE-OFFS & EDGE CASES

Message ordering in group chat:

```
  • No global total order (too expensive)
  • Per-sender ordering guaranteed
  • Client-side sorting by Snowflake ID

Large group chat (1000+ members):
  • Fan-out expensive
  • Solution: Use Kafka topic per group, consumers pull

Media messages:
  • Upload to S3, get URL
  • Send URL in message (not raw bytes over WS)
  • Thumbnail generation async

E2E Encryption:
  • Server never sees plaintext (Signal Protocol)
  • Key exchange at connection time
  • Cannot do server-side search then

■ "WebSocket for real-time, Cassandra for storage,
  Kafka for fan-out, Redis for presence"
```

I. INTERVIEW TIPS

✓ Choose WebSocket and explain why (vs polling)

✓ Address message ordering (Snowflake IDs)

✓ Explain stateful server challenge + service registry

✓ Discuss offline handling + push notifications

✓ Cover online presence with heartbeat + TTL

COMMON MISTAKES:

✗ Using HTTP for real-time messaging

✗ Not addressing server-to-server message routing

✗ Ignoring offline users

```
MNEMONIC: "WCKRP" → WebSocket, Cassandra, Kafka, Redis-presence, Push
```

10 Search Autocomplete (Typeahead)

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Return top-k suggestions as user types each character
• Examples: Google search bar, Amazon product search
• Must be very fast (<100ms per keystroke)
• Suggestions ranked by frequency/relevance

B. REQUIREMENTS

FUNCTIONAL:

• Return top 5-10 suggestions per prefix
• Update suggestions based on trending searches
• Support multiple languages
• Filter inappropriate content

NON-FUNCTIONAL:

• 10M DAU, avg 10 searches/day = 100M queries/day

```
• 5 keystrokes per query avg → 500M autocomplete requests/day
• <100ms response time (P99)
• Suggestions refresh daily or hourly
• High read:write ratio (1000:1)
```

C. TRIE DATA STRUCTURE ■

TRIE (Prefix Tree):

root
/ \
b c
/ \ \
e r a
| \ |
a e r

```
          |    (beer) |
         (bear)     (car)

Each node = one character
Path from root to node = prefix
At each node: store top-k suggestions (precomputed!)

NAIVE: traverse from prefix node → collect all completions
  O(p + n) where p = prefix length, n = subtree size

OPTIMIZED: store top-k at each node ← KEY INSIGHT
  O(p) lookup → just read stored list at prefix node!
  Trade-off: more storage (each node caches top-k)

STORAGE ESTIMATE:
  10M unique queries × avg 5 chars × 25 bytes = ~1GB
  Manageable in memory (Redis or in-process)
```

D. HIGH-LEVEL DESIGN

DATA COLLECTION:

```
  User search logs → Kafka → Aggregation job (Spark/Flink)
  → Update query frequency count
  → Rebuild trie periodically (every hour/day)

SERVING:
  Client → [CDN] → API Server → Trie Service
                                     ↓
                                Redis (trie cache)
                                     ↓ (miss)
                                Trie Service in-memory

TWO SERVICES:
  1. Data Aggregation Service (batch/streaming)
     → aggregates search logs, computes top queries per prefix
  2. Query Service (read path)
     → ultra-low latency, cached responses
```

E. TRIE STORAGE & UPDATES

STORAGE OPTIONS:

```
  Option 1: In-memory trie per service instance
    + Fastest (no network)
    - Memory limited, can't share across instances
  Option 2: Redis (distributed hash of prefix→suggestions)
    + Shared across servers, easy scaling
    - Network hop (~1ms)
  Option 3: Cassandra/DynamoDB (prefix as partition key)
    + Scalable, persistent
    - Higher latency (~10ms)

TRIE UPDATE STRATEGY:
  Daily batch rebuild:
    1. Aggregate last 7 days of search logs
    2. Build trie → serialize → push to object store (S3)
    3. Trie servers load new snapshot at low-traffic time
    4. Blue-green deployment for zero downtime

INCREMENTAL UPDATES (for real-time trending):
  Stream logs → Flink → update frequency counts
  Rebuild top-k at affected nodes
  Push updates to trie servers (async)
```

F. OPTIMIZATIONS

CLIENT-SIDE CACHING:

```
  Cache prefix→suggestions in browser localStorage
  Avoid repeated calls for same prefix

DEBOUNCING:
  Don't fire request on every keystroke
  Wait 150-300ms after last keystroke
  Reduces API calls by ~5-10x

AJAX vs GraphQL:
  Simple GET /autocomplete?q={prefix}&limit=10
  Response: ["apple", "apple store", ...]

FILTERING:
  • Remove profanity (blocklist filter at prefix level)
  • User-specific personalization: rank by user's past queries
  • A/B test ranking algorithms

MULTI-LANGUAGE:
  • Separate trie per language/locale
  • Detect language from browser headers or user settings

BROWSER TRICK:
  Use <datalist> HTML element for simple cases
  Custom dropdown for more control
```

G. SCALING

SHARDING TRIE:

```
  By first character: a-f → Shard1, g-l → Shard2...
  Problem: skewed distribution (many words start with 's')
  Better: consistent hashing on prefix

CACHING STRATEGY:
  • Top 1000 prefixes cached in CDN
  • "ap", "ap " → massive traffic
  • Long-tail prefixes served from Redis/DB

SCALE NUMBERS:
  5 chars/query × 100M queries = 500M requests/day
  = 5800 req/sec average
  = ~50K req/sec peak (10x)
  → 10-50 API servers with 10K QPS each → manageable

■ "Cache top-1000 prefixes at CDN edge for free latency win"
```

H. INTERVIEW TIPS

✓ Draw the trie and explain O(p) lookup with precomputed top-k
✓ Separate data collection from query serving
✓ Mention client-side caching + debouncing
✓ Address trie update strategy (batch vs streaming)
✓ Discuss filtering and personalization

■ MUST SAY IN INTERVIEW:

"The key insight is storing top-k suggestions

```
  at each trie node — O(1) lookup instead of O(n) traversal"

MNEMONIC: "TOP-K" → Trie, O(p)-lookup, Precompute, Keys-cached, ...
```

11 YouTube / Video Streaming

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Upload, store, process, and stream videos at massive scale
• Examples: YouTube, Netflix, TikTok, Twitch
• Challenges: encoding (multi-resolution), CDN, storage cost, live streaming
• 500 hours of video uploaded to YouTube per minute!

B. REQUIREMENTS

FUNCTIONAL:

• Upload video (up to 1GB, up to 4K)
• Stream video (adaptive bitrate)
• Like, comment, subscribe
• Search videos
• Recommendations (out of scope)

NON-FUNCTIONAL:

• 5M DAU uploading, 2B DAU watching
• Video available within 5 min of upload
• Smooth playback (adaptive bitrate)
• 99.9% availability
• Global reach

ESTIMATION:

• 500hr/min uploaded = 30K sec video/min

```
• 1 min 1080p ≈ 350MB → 500hr = 10.5TB/min upload
• Storage: petabytes (multiple resolutions + formats)
```

C. VIDEO UPLOAD & PROCESSING

UPLOAD FLOW:

```
  User → [Presigned S3 URL] → Upload directly to S3
              ↓ (S3 event notification)
        [Video Processing Queue] (Kafka/SQS)
              ↓
        [Transcoding Workers] (EC2 fleet)
              ↓
  Output: Multiple resolutions (360p, 720p, 1080p, 4K)
          Multiple formats (H.264, H.265, VP9, AV1)
              ↓
        [CDN Origin (S3)] → [CDN Edge Nodes]

WHY PRESIGNED URL?
  • Client uploads directly to S3 (bypass app server)
  • App server only issues time-limited signed URL
  • Saves bandwidth cost on app server

TRANSCODING:
  • Segment video into 6-10 sec chunks (HLS/DASH)
  • Process chunks in parallel (faster)
  • DAG of tasks: split → transcode_720p, transcode_1080p, ... → merge
  • Use GPU instances for faster encoding
```

D. VIDEO STREAMING

ADAPTIVE BITRATE STREAMING (ABR) ■:

```
  • HLS (HTTP Live Streaming) - Apple
  • DASH (Dynamic Adaptive Streaming over HTTP)

  How it works:
  1. Video split into small segments (2-10 sec each)
  2. Each segment encoded at multiple bitrates
  3. Manifest file (.m3u8/.mpd) lists available segments
  4. Player monitors bandwidth → selects appropriate bitrate
  5. Switches quality mid-stream seamlessly

  BITRATE LADDER:
    360p: 500kbps
    720p: 2.5Mbps
    1080p: 5Mbps
    4K: 15-25Mbps

CDN STRATEGY:
  • Videos served from CDN edges (not origin)
  • Popular videos: pre-warmed in CDN
  • Long-tail videos: served from S3/origin
  • Use consistent hashing for CDN server selection
```

E. DATABASE DESIGN

VIDEO METADATA (MySQL/PostgreSQL):

```
  video_id    BIGINT      PK (Snowflake)
  user_id     BIGINT      FK
  title       VARCHAR(500)
  description TEXT
  status      ENUM        (processing/ready/failed)
  duration_sec INT
  view_count  BIGINT
  like_count  BIGINT
  created_at  TIMESTAMP

VIDEO_SEGMENTS (S3 + manifest):
  s3://videos/{video_id}/720p/segment_001.ts
  s3://videos/{video_id}/manifest.m3u8

USER_INTERACTIONS (Cassandra/NoSQL):
  video_id, user_id, interaction_type (view/like/comment)
  → High write throughput, denormalized for reads

COMMENTS (Cassandra):
  video_id (PK), comment_id (clustering key, desc)
  → Efficient: fetch latest N comments per video
```

F. SCALING

UPLOAD BOTTLENECK:

```
  • Parallel chunk upload (split large video, upload chunks concurrently)
  • Resume upload on failure (track uploaded chunks in Redis)

PROCESSING BOTTLENECK:
  • Auto-scaling transcoding workers (EC2 Spot instances for cost)
  • Prioritize short videos (upload queue with priority)
  • Skip 4K transcoding for short-tail content

VIEW COUNT ACCURACY:
  • Problem: 1B views/day → can't increment DB on every view
  • Solution: Buffer counts in Redis → batch write to DB every 60s
  • Use Redis INCR (atomic) → persist to DB with cronjob

STORAGE COST:
  • Cold storage (S3 Glacier) for videos < 1000 views
  • Hot storage (S3 Standard) for popular videos
  • Compress old formats (migrate to AV1 for ~30% savings)
```

G. INTERVIEW TIPS

✓ Mention presigned S3 URL for direct upload

✓ Explain adaptive bitrate streaming (HLS/DASH)

✓ Discuss parallel transcoding with DAG

✓ Address CDN caching strategy

✓ Cover view count accuracy with Redis buffering

■ MUST SAY IN INTERVIEW:

"Segment video into chunks, transcode in parallel to multiple bitrates, serve via CDN with adaptive streaming"

```
MNEMONIC: "UPSET" → Upload-presigned, Processing-DAG, Segment-HLS,
  Edge-CDN, Transcode-GPU
```

12 Google Drive / File Storage

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Cloud file storage with sync, share, version history

• Examples: Google Drive, Dropbox, OneDrive, Box

• Challenges: sync conflict, large file chunking, offline access, sharing

B. REQUIREMENTS

FUNCTIONAL:

• Upload/download files (any type)

• File sync across devices

• Share files/folders with permissions

• Version history (last 30 versions)

• Offline access with sync on reconnect

NON-FUNCTIONAL:

• 50M DAU, avg 10GB per user

• Max file size: 10GB

• P99 upload < 30sec for 100MB file

• 99.99% data durability

• Eventual consistency for sync

ESTIMATION:

• 50M users × 10GB = 500PB storage

• Daily uploads: 10M files × 1MB avg = 10TB/day

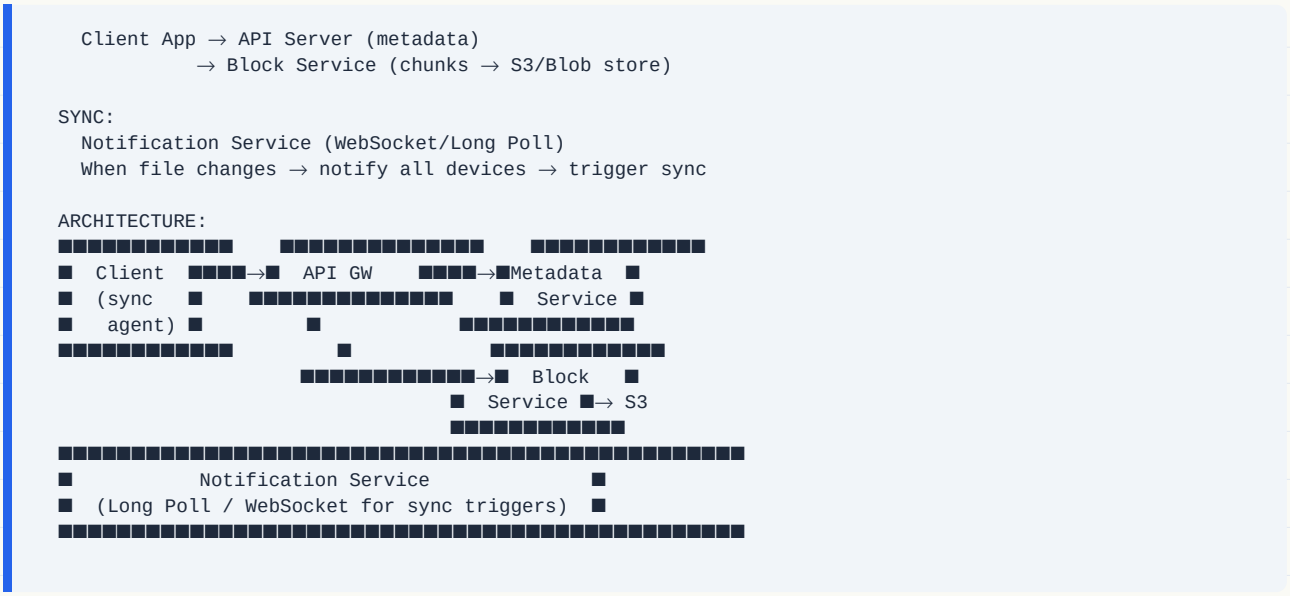
C. FILE CHUNKING ■

WHY CHUNK FILES?

```
  • Resume broken uploads (only re-upload failed chunks)
  • Delta sync: only upload changed chunks
  • Parallel upload (multiple chunks simultaneously)
  • Deduplication: if chunk exists (same hash), don't re-upload

CHUNK SIZE: 4MB (trade-off: too small = too many API calls)

CHUNKING FLOW:
  Client splits file into 4MB chunks
  For each chunk: hash = SHA-256(chunk_bytes)
  Check server: does this hash exist? (dedup)
  Upload only new chunks → commit file with list of chunk hashes

CONTENT-DEFINED CHUNKING (Dropbox):
  Split at content-defined boundaries (Rabin fingerprint)
  Better dedup when content inserted/deleted mid-file
  Fixed chunking: insertion shifts all subsequent chunk boundaries

DELTA SYNC:
  User edits file → only modified chunks differ
  Client computes chunk hashes → sends diff to server
  Server updates only changed chunks
```

D. HIGH-LEVEL DESIGN

UPLOAD:



E. DATABASE DESIGN

METADATA DB (MySQL/PostgreSQL):

```
files:
  file_id     BIGINT      PK
  user_id     BIGINT      FK
  name        VARCHAR
  size_bytes  BIGINT
  checksum    VARCHAR(64) (SHA-256 of full file)
  created_at  TIMESTAMP
  updated_at  TIMESTAMP
  is_deleted  BOOLEAN

file_versions:
  version_id  BIGINT      PK
  file_id     BIGINT      FK
  chunks      JSONB       [{hash, sequence_num}]
  created_at  TIMESTAMP

chunks:
  chunk_hash  VARCHAR(64) PK  (content-addressed)
  s3_key      VARCHAR
  size_bytes  INT
  ref_count   INT  (for deduplication)

WHY CONTENT-ADDRESSED STORAGE?
  chunk_hash → S3 key
  Same chunk across users = stored once
  ref_count > 0: keep; ref_count = 0: delete (GC)
```

F. SYNC CONFLICT RESOLUTION

SCENARIO: User edits file on phone + laptop simultaneously offline

```
  → Both create new versions
  → Which wins?

STRATEGIES:
  1. Last Write Wins (LWW): simpler but loses data
  2. Conflict copies: both versions saved, user resolves (Dropbox)
  3. OT/CRDT: collaborative real-time editing (Google Docs)

VECTOR CLOCKS for detecting conflicts:
  Device A: [A:3, B:1]
  Device B: [A:2, B:2]
  → Concurrent edits → conflict
  → Create conflict copy

GOOGLE DOCS APPROACH (OT):
  Operations transformed against concurrent ops
  All users converge to same document
  Complex to implement

■ "For file sync (binary files), conflict copies.
   For collaborative text, use OT or CRDT"
```

G. SHARING & PERMISSIONS

PERMISSION MODEL:

```
  ACL (Access Control List) per file/folder:
    {file_id: 123, user_id: 456, permission: "view|edit|comment"}

SHARING MECHANISMS:
  1. Direct share: invite by email → add ACL entry
  2. Link sharing: generate token → anyone with link can access
     token → (file_id, permission, expiry)

FOLDER PERMISSIONS:
  Inherit from parent folder (recursive)
  Override possible at file level
  Check permission: walk up folder tree (expensive)
  Optimization: materialized permission cache per file

STORAGE QUOTA:
  Track per user: used_bytes
  Update on upload/delete (decrement on delete)
  Hard limit: reject uploads when quota exceeded
```

H. INTERVIEW TIPS

✓ Explain chunking + delta sync in detail
✓ Content-addressed storage for deduplication
✓ Discuss conflict resolution strategies
✓ Mention notification service for sync triggers
✓ Cover version history with chunk immutability

■ MUST SAY IN INTERVIEW:

"Files are split into 4MB chunks, content-addressed by SHA-256. Delta sync only uploads changed chunks."

```
MNEMONIC: "CHUNKS" → Content-addressed, Hash-dedup, Upload-delta,
  Notify-sync, Keep-versions, Share-ACL
```

13 Proximity Service

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Find businesses/places within radius of user's location
• Examples: Yelp (restaurants nearby), Google Maps (ATMs near me)
• Core challenge: efficient geospatial search at scale
• Not tracking moving objects (that's Nearby Friends)

B. REQUIREMENTS

FUNCTIONAL:

• Search businesses within radius (default 500m)
• Return name, address, rating, distance
• Add/update/delete business info
• Filter by category (restaurant, gas station...)

NON-FUNCTIONAL:

• 100M DAU, 5 queries/day = 500M searches/day
• <200ms search latency
• Businesses: 200M total
• High read:write ratio (1000:1)
• Business info updated infrequently

C. GEOSPATIAL INDEXING (CRITICAL)

APPROACH 1: Two-dimensional search

```
  SELECT * WHERE lat BETWEEN (user_lat ± delta)
              AND lng BETWEEN (user_lng ± delta)
  Problem: Can't use index on both lat AND lng efficiently

APPROACH 2: GEOHASH ■
  • Encode lat/lng into string
  • Grid subdivisions: each char refines by 32x
  • Length 6 → ~1.2km × 0.6km precision
  • Nearby cells share common prefix (mostly!)

  Geohash for proximity:
  1. Convert user location → geohash(length=6)
  2. Get 9 cells: center + 8 neighbors
     (important! boundary cases!)
  3. SELECT * WHERE geohash LIKE 'dp3wj%'
  4. Post-filter by exact distance

APPROACH 3: QUADTREE
  • Tree structure, recursively divide 2D space
  • Split when region has > threshold points (e.g., 100)
  • Dynamic: dense cities → deep tree, sparse areas → shallow
  • Good for in-memory indexing

APPROACH 4: PostGIS (PostgreSQL extension)
  • Full-featured geospatial SQL
  • ST_DWithin(geom, point, radius)
  • Good for smaller scale (<50M records)
```

D. HIGH-LEVEL DESIGN

```
■■■■■■■■■■■         ■■■■■■■■■■■■■■■■■■■■■■
■ Client ■■■■■■■■■■→■ Location Service■←→ Geohash Index (Redis)
■■■■■■■■■■■         ■■■■■■■■■■■■■■■■■■■■■■       ↓ (miss)
                          ↓              Business DB
                    ■■■■■■■■■■■■■■■■■■     (read replica)
                    ■Business Service■←→ Business DB
                    ■■■■■■■■■■■■■■■■■■

SEARCH FLOW:
  1. User sends lat/lng + radius
  2. Convert lat/lng → geohash (precision based on radius)
  3. Get geohash + 8 neighbors
  4. Query Redis index: SMEMBERS geohash:dp3wjm
  5. Fetch business details from cache/DB
  6. Calculate exact distance, filter, sort, return
```

E. DATABASE DESIGN

TABLE: businesses

business_id BIGINT PK

name VARCHAR

category VARCHAR

```
  lat          DECIMAL(9,6)
  lng          DECIMAL(9,6)
  geohash      VARCHAR(6)  INDEXED ← KEY FIELD
  address      TEXT
  rating       FLOAT
  is_active    BOOLEAN

REDIS GEOHASH INDEX:
  Key: geohash:{geohash_prefix}
  Type: Set of business_ids
  SMEMBERS geohash:dp3wjm → [biz123, biz456, ...]

ALTERNATIVE: Redis Geo commands
  GEOADD locations lng lat member
  GEORADIUS locations lng lat radius km
  → Built-in geohash under the hood

UPDATE STRATEGY:
  Business location changes (rare):
  1. Remove from old geohash set
  2. Recalculate new geohash
  3. Add to new geohash set
  4. Update business DB
```

F. RADIUS CALCULATION

HAVERSINE FORMULA (exact distance on sphere):

```
  d = 2R × arcsin(√(sin²(Δlat/2) + cos(lat1)×cos(lat2)×sin²(Δlng/2)))
  Expensive for large result sets

OPTIMIZATION:
  1. Use geohash for rough filtering (fast, approximate)
  2. Apply haversine on filtered set (~100-1000 businesses)
  3. Geohash → bounding box filter → haversine

GEOHASH PRECISION vs RADIUS:
  1 char → ~5000km × 5000km
  4 chars → ~39km × 20km
  6 chars → ~1.2km × 0.6km ← typical for local search
  8 chars → ~38m × 19m

DYNAMIC RADIUS:
  User specifies 1km → use geohash length 6
  User specifies 10km → use geohash length 5
  If too few results → expand search to larger geohash prefix
```

G. SCALING

READ SCALING:

```
  • Redis cluster for geohash index (hot reads)
  • Read replicas for business detail DB
  • Cache popular business details (CDN for images)

WRITE SCALING:
  • Business updates are rare (use write-through cache)
  • New business added → update both DB and Redis index
  • ~100 business updates/day globally → no write concern

GEOGRAPHIC SHARDING:
  Shard by geographic region (NA, EU, APAC)
  Users query their regional shard
  Reduces cross-region latency

■ "Geohash is the key insight: converts 2D problem
  to 1D prefix matching, enabling efficient indexing"
```

H. INTERVIEW TIPS

```
✓ Explain why 2D indexing is hard → lead to geohash
✓ Always mention 8-neighbor cells for boundary problem
✓ Two-step: geohash filter → exact haversine calculation
✓ Discuss radius-to-geohash-precision mapping
✓ Mention expanding search if too few results

MNEMONIC: "GEO" → Geohash-index, Eight-neighbors, Only-haversine-at-end
```

14 Nearby Friends

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Show friends who are physically close to you in real-time
• Example: Facebook Nearby Friends
• Different from Proximity Service: locations are MOVING (dynamic)
• Challenge: high-frequency location updates + efficient querying

B. REQUIREMENTS

FUNCTIONAL:

• Show friends within 5 miles
• Update location every 30 seconds
• Opt-in only (privacy)
• Show distance, not exact location

NON-FUNCTIONAL:

• 1B users, 10% opted in = 100M active
• 30% of opted-in online = 30M concurrent
• Location update: every 30 sec = 1M updates/sec
• <1 sec to see friend's new location
• Highly available (AP)

C. HIGH-LEVEL DESIGN

Location Update Flow:

```
User App → [Location Update API] → [Location Service]
                                          ↓
                                   [Redis] (current location)
                                          ↓
                                   [Kafka] (location events)
                                          ↓
                            ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                            ■ Location History Service   ■
                            ■    (Cassandra)             ■
                            ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Nearby Query:
  User App → WebSocket → [Nearby Service]
  [Nearby Service] pulls friend locations → computes distances
  → pushes updates to user via WebSocket

■ KEY: Pub/Sub per user
  User A moves → publish to channel "location:{user_a_id}"
  User B (friend of A) subscribes to channel
  → B's server gets update → push to B via WebSocket
```

D. LOCATION STORAGE

CURRENT LOCATION (Redis):

```
  Key: location:{user_id}
  Value: {lat: 37.7, lng: -122.4, ts: 1686000000}
  TTL: 120 seconds (expire if no update = user offline)

LOCATION HISTORY (Cassandra):
  TABLE: location_history
    user_id    BIGINT   PARTITION KEY
    timestamp  BIGINT   CLUSTERING KEY DESC
    lat        FLOAT
    lng        FLOAT

FRIEND LIST (Redis or DB):
  user:{id}:friends → Set of friend_ids
  (pre-computed, cached for fast access)

GEOHASH FOR GROUPING:
  Optional: group users by geohash cell
  Nearby query: check users in same + adjacent cells
  Updates shared via Pub/Sub per geohash cell
```

E. SCALING CHALLENGES

1M UPDATES/SEC:

```
  • Kafka for ingestion (handles 1M+ msg/sec)
  • Multiple Kafka partitions (partition by user_id)
  • Consumer services update Redis

PUB/SUB SCALING:
  • Redis Pub/Sub (simple but limited)
  • For scale: Erlang-based (WhatsApp), or custom
  • Option: WebSocket server subscribes to Kafka
    → Each user's WebSocket server consumes friend's events

FRIEND LIST CHALLENGE:
  • 500 friends × 1M updates/sec = 500M pub/sub events/sec
  • Solution: Only subscribe when user opens Nearby feature
  • Or: Batch updates (send diffs every 30 sec, not real-time)

PRIVACY:
  • Never store exact location in DB (fuzzy location)
  • Show "within 2 miles" not exact coordinates
  • Delete location history after 7 days
```

F. INTERVIEW TIPS

✓ Distinguish from static proximity (moving locations)

```
✓ Pub/Sub model: each user location update → notify relevant friends
✓ Redis for current locations (with TTL for offline detection)
✓ Kafka for high-throughput location ingestion
✓ Address privacy (fuzzy location, opt-in)

■ MUST SAY: "1M updates/sec → Kafka for ingestion,
  Redis for current state, WebSocket for push to clients"

MNEMONIC: "KRCW" → Kafka, Redis-TTL, Channel-pubsub, WebSocket
```

15 Google Maps

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Navigation service: directions, ETA, traffic-aware routing

• Features: map tiles, turn-by-turn, live traffic, search

• Most complex system design problem (many sub-systems)

• Focus on: routing, map tiles, location data

B. REQUIREMENTS

FUNCTIONAL:

• Render map tiles (visual map)

```
  • Point A → Point B routing with ETA
  • Turn-by-turn navigation
  • Real-time traffic updates
  • Search (address, POI)

NON-FUNCTIONAL:
  • 1B DAU
  • <1 sec for route calculation
  • Map tiles load fast (<100ms)
  • Traffic updates every minute
  • High availability
```

C. MAP DATA REPRESENTATION

ROAD NETWORK AS GRAPH:

```
    Node = intersection (lat/lng)
    Edge = road segment (distance, speed_limit, restrictions)
    Edge weight = travel time (function of distance + traffic)

  MAP TILE SYSTEM:
    • World divided into tiles at each zoom level
    • Zoom 0: 1 tile (whole world)
    • Zoom n: 4^n tiles (quadtree structure)
    • Each tile = 256×256 PNG image
    • Rendered offline, stored in S3/CDN

  TILE SERVING:
    URL: /tiles/{zoom}/{x}/{y}.png
    CDN caches tiles (static content)
    Pre-rendered for all zoom levels

  VECTOR TILES (modern):
    • Store geometric shapes (not images)
    • Rendered on device (client-side)
    • Smaller transfer, dynamic styling
    • Google Maps migrated to vector tiles
```

D. ROUTING ALGORITHM

DIJKSTRA'S ALGORITHM:

• Finds shortest path in weighted graph

```
  • O((V + E) log V) with priority queue
  • Too slow for full road network (100M+ nodes)

A* ALGORITHM (heuristic):
  • Dijkstra + heuristic (straight-line distance to dest)
  • Explores fewer nodes → faster
  • Still slow for cross-country routes

BIDIRECTIONAL DIJKSTRA:
  • Search from source AND destination simultaneously
  • Meet in middle → ~√V nodes each
  • 2-4x faster than unidirectional

HIERARCHICAL ROUTING (Google's approach) ■:
  • Preprocess: identify "highway" nodes (important junctions)
  • Routing layers:
    L1: local roads (within city)
    L2: arterial roads
    L3: highways
  • For long routes: go up hierarchy, route at highway level, come down
  • Contraction Hierarchies (CH) → 1000x speedup!

ETA CALCULATION:
  Base: distance / speed_limit
  Adjust: historical traffic by time-of-day
  Real-time: subtract/add based on current traffic events
```

E. REAL-TIME TRAFFIC

DATA SOURCES:

```
  • GPS data from user phones (passively collected)
  • Traffic sensors, cameras
  • Incident reports (user-submitted)
  • Historical patterns (baseline)

PROCESSING:
  GPS pings → Kafka → Stream Processor (Flink)
  → Map-match GPS to road segments
  → Calculate current speed per segment
  → Compare to free-flow speed → congestion level
  → Update routing weights every 60 sec

MAP MATCHING:
  Raw GPS → nearest road segment assignment
  Handles GPS noise (Kalman filter or HMM)
  "I'm on I-280 not the frontage road"

TRAFFIC DISTRIBUTION:
  Edge updates → Push to routing service cache
  Routing service recalculates ETAs
  Navigate users away from congestion
```

F. LOCATION SERVICES

```
GEOCODING: Address → (lat, lng)
  "1600 Amphitheatre Pkwy" → (37.422, -122.084)
  Database: address → coordinates

REVERSE GEOCODING: (lat, lng) → Address
  Voronoi diagram or R-tree for nearest address lookup

SEARCH:
  Elasticsearch for POI search
  Combine: text relevance + location proximity
  "pizza near me" → text: pizza + location: user coords

POSITIONING:
  GPS: accurate outdoors (±3m)
  Cell tower triangulation: indoors, less accurate (±100m)
  WiFi positioning: ±15m
  Dead reckoning: estimate from last known + speed + direction
```

G. DATABASE ARCHITECTURE

MAP DATA:

```
  PostgreSQL + PostGIS (road network, POIs)
  Or: Specialized graph DB (Neo4j) for routing

TILES:
  S3 for tile storage
  CloudFront/CDN for tile delivery
  Redis cache for hot tiles

TRAFFIC:
  Cassandra for time-series traffic data
  Redis for current speed per road segment

USER DATA:
  DynamoDB for user preferences, saved places
  Audit trails for navigation history

SCALE: petabytes of map data, TB of traffic data/day
```

H. INTERVIEW TIPS

✓ Start with graph representation of road network

```
✓ Explain why Dijkstra alone fails at scale → hierarchical routing
✓ Separate: tile serving (CDN) from routing (compute)
✓ Real-time traffic: GPS data → Kafka → stream processing
✓ Mention ETA = historical + real-time adjustment

■ MUST SAY: "Contraction Hierarchies for routing,
  vector tiles via CDN, Kafka for traffic stream processing"

COMMON MISTAKES:
  ✗ Only mentioning Dijkstra (too slow at scale)
  ✗ Not discussing map tiles separately from routing
  ✗ Ignoring GPS data collection for traffic

MNEMONIC: "GRAPH-TILE-TRAFFIC" → key three concerns
```

16 Distributed Message Queue

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Async communication between producers and consumers
• Examples: Apache Kafka, Amazon SQS, RabbitMQ, Pulsar
• Decouples systems, enables buffering, replay, fan-out
• Kafka: log-based, immutable, consumer controls offset
• SQS: message deleted after ack (queue semantics)

B. REQUIREMENTS

FUNCTIONAL:

• Producer: send message to topic
• Consumer: poll/subscribe to topic
• Message retention: 7-14 days
• At-least-once delivery
• Message ordering (within partition)

NON-FUNCTIONAL:

• Throughput: 1M messages/sec

```
• Low latency: <10ms producer → consumer
• High availability: 99.99%
• Durable: messages persisted to disk
• Scalable: add brokers without downtime
```

C. KAFKA ARCHITECTURE (DEEP DIVE) ■

```
TOPIC → PARTITIONS → REPLICAS:
  Topic: logical channel (e.g., "user_events")
  Partition: physical ordered log (append-only)
  Replica: copy of partition on different broker

KAFKA CLUSTER:
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■  Topic: orders (3 partitions)        ■
  ■  Partition 0: [msg1, msg3, msg5]    ■ → Broker1 (leader)
  ■  Partition 1: [msg2, msg4, msg6]    ■ → Broker2 (leader)
  ■  Partition 2: [msg7, msg8, msg9]    ■ → Broker3 (leader)
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Each partition replicated on 2-3 brokers

PRODUCER:
  Key (optional) → hash → partition selection
  Same key → same partition → ordering guaranteed
  No key → round-robin → better distribution

CONSUMER GROUP:
  Group of consumers sharing topic consumption
  Each partition → exactly ONE consumer in group
  Consumers = Partitions → max parallelism

OFFSET:
  Each message has monotonically increasing offset
  Consumer tracks its offset → can replay
  Commit offset → "I've processed up to here"
```

D. MESSAGE RETENTION & STORAGE

DISK STORAGE (Sequential Writes):

```
  Kafka writes to disk sequentially (not random)
  OS page cache → disk throughput ~600MB/s
  "Kafka is fast because disk is fast when sequential"

LOG SEGMENTS:
  Partition = multiple segment files
  Active segment: appended to
  Older segments: read-only, compacted/deleted based on policy

RETENTION POLICIES:
  Time-based: delete messages older than 7 days
  Size-based: delete when partition exceeds 1GB
  Compaction: keep only latest value per key (like a KV store)

REPLICATION:
  Leader handles reads+writes
  Followers replicate asynchronously
  ISR (In-Sync Replicas): replicas caught up with leader
  ACK levels:
    acks=0: no wait (fastest, can lose data)
    acks=1: leader written (balanced)
    acks=all: all ISR written (safest) ← recommended
```

E. EXACTLY-ONCE SEMANTICS

```
AT-MOST-ONCE: Producer doesn't retry → messages may be lost
AT-LEAST-ONCE: Producer retries → messages may be duplicated
EXACTLY-ONCE: ■ Hardest to achieve

KAFKA EXACTLY-ONCE (since 0.11):
  Producer side: idempotent producer
    Each message has (producer_id, sequence_num)
    Broker deduplicates: ignore sequence already seen

  Consumer+Producer (transactional):
    Process message + produce result in one transaction
    READ → PROCESS → WRITE atomically
    Offset commit + output write in same transaction

PRACTICAL APPROACH:
  Most systems use at-least-once + idempotent consumers
  Consumer checks: "have I processed message_id X?"
  If yes: skip (dedup table in Redis/DB)
```

F. SCALING

THROUGHPUT SCALING:

```
  Add partitions → more parallelism
  Add consumer instances (= partition count)
  Add brokers → more partition leaders

PRODUCER BATCHING:
  Buffer messages → send in batches (linger.ms = 5ms)
  Reduces TCP round trips, better compression

COMPRESSION:
  LZ4: fast, moderate compression (default)
  Snappy: balanced
  GZIP: high compression, slower

MULTI-DATACENTER (MirrorMaker):
  Replicate Kafka topics across DCs
  Active-Active or Active-Passive
  Useful for DR and geo-distribution

ZOOKEEPER → KRAFT (KIP-500):
  Old: ZooKeeper for metadata management
  New: Kafka Raft (KRaft) built-in consensus
  Eliminates ZooKeeper dependency
```

G. INTERVIEW TIPS

✓ Explain partition-based parallelism

✓ Consumer group = each partition consumed by one consumer

✓ Discuss offset management (commit strategies)

✓ acks=all for durability, ISR for replication

✓ Address exactly-once with idempotent producer

■ MUST SAY IN INTERVIEW:

"Kafka = distributed append-only log.

Partition by key for ordering, consumer groups for parallelism,

acks=all + ISR for durability"

COMPARE: Kafka vs SQS

```
  Kafka: retain messages, replay, stream processing
  SQS: simple queue, delete on ack, auto-scaling, managed

MNEMONIC: "TOPIC-PART-CONSUMER" → three key abstractions
```

17 Metrics Monitoring & Alerting

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Collect, store, visualize, and alert on system metrics
• Examples: DataDog, Prometheus + Grafana, CloudWatch, New Relic
• Metrics types: counters, gauges, histograms, summaries
• Use: SLO/SLA tracking, anomaly detection, capacity planning

B. REQUIREMENTS

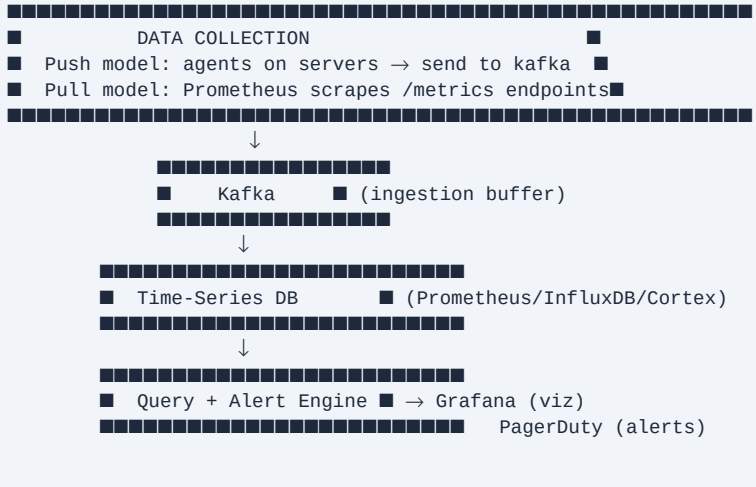
FUNCTIONAL:

• Collect metrics from 1000s of servers
• Store time-series data
• Query: raw + aggregated metrics
• Alert: threshold, anomaly, rate-of-change
• Dashboard visualization

NON-FUNCTIONAL:

• 100M metrics/min ingestion
• Query latency < 1 sec for dashboards
• 1 year retention (downsample over time)
• High availability (lose metrics < lose money)

C. HIGH-LEVEL DESIGN



D. TIME-SERIES DATABASE

WHY SPECIAL DB? (not RDBMS):

```
  • High write throughput (millions of points/min)
  • Time-based queries (last 1hr, time range)
  • Auto-downsample old data
  • High compression (delta encoding, Gorilla)

GORILLA COMPRESSION (Facebook):
  • Store delta of timestamps (1-bit if same interval)
  • XOR of consecutive values (most bits same → compress well)
  • 12x compression vs raw floats

POPULAR TSDB:
  Prometheus: pull-based, single node, 15-day retention
  Cortex/Thanos: Prometheus + long-term storage + HA
  InfluxDB: SQL-like, push/pull, long retention
  TimescaleDB: PostgreSQL extension
  Druid: analytics TSDB (for ad-tech scale)

DATA MODEL:
  metric_name{label1=v1, label2=v2} value timestamp
  Example:
    http_requests_total{method="GET", status="200"} 1234 1686000000

DOWNSAMPLING:
  Raw: 15-second resolution, keep 15 days
  1-minute: keep 1 month
  1-hour: keep 1 year
  1-day: keep 5 years
```

E. ALERTING SYSTEM

ALERT RULE:

```
  Condition: avg(http_error_rate[5m]) > 0.01 (1%)
  Trigger: alert fires when condition true for X minutes
  Notify: PagerDuty, Slack, Email, OpsGenie

ALERT TYPES:
  Threshold: metric > static value
  Anomaly: deviation from ML baseline
  Rate-of-change: sudden spike/drop
  SLO burn rate: consuming error budget too fast

ALERT FATIGUE:
  • Too many alerts → on-call ignores them (crying wolf)
  • Solution:
    1. Alert on symptoms (user-facing) not causes
    2. Group related alerts
    3. Define severity (P1=page, P2=ticket, P3=log)
    4. Auto-resolve when condition clears

SLO ALERTING (Google SRE approach):
  Error budget = 1 - SLO target
  If burning >5x budget rate → alert immediately
  If burning >2x → alert within 1 hour
  "Burn rate alerting" = more nuanced than threshold
```

F. METRICS COLLECTION MODELS

PUSH MODEL (DataDog, InfluxDB):

Agents on each host push metrics to collector

+ Works behind firewalls

+ Agent can buffer during network blip

- Agent management overhead

PULL MODEL (Prometheus):

Prometheus scrapes /metrics on each service

+ Service controls what it exposes

```
    + Health check built in (if can't scrape → down)
    - Must have network access to all services
    - Scaling: many targets → many scrapes

  METRICS TYPES:
    Counter: only increases (total requests)
    Gauge: can go up/down (memory usage)
    Histogram: distribution of values (request latency buckets)
    Summary: pre-calculated percentiles
```

G. INTERVIEW TIPS

✓ Cover 4 metric types (counter, gauge, histogram, summary)

✓ Explain pull vs push model trade-offs

✓ Mention Gorilla compression for storage efficiency

✓ Discuss alert fatigue and SLO-based alerting

✓ Address long-term storage with downsampling

■ MUST SAY IN INTERVIEW:

"Use Kafka as ingestion buffer to smooth write spikes. TSDB with Gorilla compression. Alert on symptoms, not causes. Burn-rate alerting for SLOs."

```
  MNEMONIC: "CIPAS" → Collect, Ingest(Kafka), Persist(TSDB),
    Alert, Serve(Grafana)
```

18 Ad Click Aggregation

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Count ad clicks accurately for billing and reporting
• Examples: Google AdWords, Facebook Ads, Display advertising
• Two modes: real-time dashboard + batch reconciliation
• Must be accurate (money is involved!)
• Scale: 10B clicks/day (Facebook-scale)

B. REQUIREMENTS

FUNCTIONAL:

• Count clicks per ad_id in last M minutes
• Top N ads by clicks in last M minutes
• Filter by region, device, user demographics
• Hourly/daily aggregate reports

NON-FUNCTIONAL:

• 10B clicks/day = ~116K clicks/sec average
• Peak 3x: ~350K clicks/sec
• Query latency <1 sec for dashboards
• <1% error rate in counting
• At-least-once processing (no lost clicks)
• Data retention: 7 years (legal requirement)

C. HIGH-LEVEL DESIGN

CLICK EVENT FLOW:

```
Ad Click → [API Servers] → [Kafka] → [Stream Processor]
                                          ↓
                                  [OLAP DB / Aggregation Store]
                                          ↓
                                  [Dashboard / Reporting API]

BATCH RECONCILIATION (for accuracy):
Raw Logs → [Data Lake (S3)] → [Spark Batch Job] → [Reconciliation]
                                                        ↓
                                                  Compare with stream results
                                                  Fix discrepancies

DUAL PIPELINE:
  Stream: real-time counts (fast but approximate)
  Batch: exact counts (slow but accurate)
  Reconciliation: correct stream errors with batch
```

D. AGGREGATION STRATEGIES

APPROACH 1: Real-time (Flink/Kafka Streams):

```
  Tumbling windows: 1-min, 5-min, 1-hour buckets
  State: Map<ad_id, count> per window
  Output: write aggregates to DB every N seconds

APPROACH 2: Lambda Architecture:
  Speed layer (stream): low latency, approximate
  Batch layer (Spark): accurate, slower
  Serving layer: merge both results
  Problem: complex, maintain two codebases

APPROACH 3: Kappa Architecture:
  Only stream processing
  Replay Kafka for historical reprocessing
  Simpler but requires long Kafka retention

■ "Use Flink tumbling windows for real-time,
  Spark batch for reconciliation and billing"

DEDUP IN CLICK EVENTS:
  Same click sent twice (retry) → double-count
  Solution: unique click_id per event
  Flink state: seen_click_ids per window
  Or: probabilistic dedup (Bloom filter)
```

E. DATABASE DESIGN

RAW EVENTS (Data Lake):

```
  S3: ad_click/{year}/{month}/{day}/{hour}/part_*.parquet
  Schema: click_id, ad_id, user_id, timestamp, ip, region, device

AGGREGATED COUNTS (ClickHouse/Druid):
  TABLE: ad_click_aggregates
    ad_id       BIGINT
    window_start TIMESTAMP  (minute/hour granularity)
    click_count BIGINT
    unique_users BIGINT
    regions     MAP<string, int>

SERVING (Redis + Materialized Views):
  Redis: ad:{id}:clicks:5min → INCR every click (buffered)
  Flush to DB every 60 sec

TOP-N ADS:
  Heap of size N, updated per window
  Or: Redis Sorted Set ZADD ad_clicks score ad_id
  → ZREVRANGE ad_clicks 0 N-1 (top N)
```

F. FAULT TOLERANCE

KAFKA:

```
  Retain raw clicks for 7 days (reprocess on failure)
  Multiple consumer groups (one for real-time, one for batch)

FLINK STATE:
  Checkpoints every 30 sec → resume from last checkpoint
  Exactly-once with Kafka transactional consumer

LATE ARRIVALS:
  Click happens at T but arrives at T+5 min (network delay)
  Flink watermark: "events older than W are late"
  Late events: update aggregate if within grace period
  After grace: drop or send to separate "late events" stream

HOTSPOT ADS:
  Super popular ad → single Kafka partition → bottleneck
  Solution: partition by ad_id + random suffix
  Aggregate shards at read time
```

G. INTERVIEW TIPS

✓ Dual pipeline: stream (fast) + batch (accurate)

✓ Address deduplication (unique click_id)

✓ Flink windowing: tumbling vs sliding vs session

✓ Late event handling with watermarks

✓ Hot partition problem for viral ads

■ MUST SAY IN INTERVIEW:

"Stream for real-time, batch for reconciliation.
Never trust stream counts for billing—reconcile with batch."

```
MNEMONIC: "KAFKA-FLINK-SPARK" → Ingest, Stream-agg, Batch-reconcile
```

19 Hotel Reservation System

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Book hotel rooms with inventory management
• Examples: Booking.com, Expedia, Airbnb
• Key challenge: prevent double-booking (inventory concurrency)
• Revenue-critical: every lost booking = lost money

B. REQUIREMENTS

FUNCTIONAL:

• Search available hotels/rooms for dates + location
• View hotel/room details
• Book a room (reserve specific dates)
• Cancel reservation
• View my reservations

NON-FUNCTIONAL:

• 5000 hotels, 1M rooms total
• 10:1 read:write
• Peak: Black Friday / holidays
• No double booking (critical!)
• Idempotent booking (no duplicate charges)
• < 1 sec for search results

C. HIGH-LEVEL DESIGN

D. INVENTORY MANAGEMENT (CRITICAL)

TABLE: room_inventory
hotel_id BIGINT
room_type ENUM
date DATE
total_rooms INT
reserved INT

```
  PRIMARY KEY (hotel_id, room_type, date)

AVAILABILITY CHECK:
  SELECT (total_rooms - reserved) AS available
  WHERE hotel_id=X AND room_type=Y AND date BETWEEN check_in AND check_out-1

BOOKING TRANSACTION:
  BEGIN TRANSACTION
    -- Lock the row(s)
    SELECT total_rooms, reserved FROM room_inventory
    WHERE ... FOR UPDATE  ← pessimistic locking
    -- Check availability
    IF reserved + count <= total_rooms:
      UPDATE room_inventory SET reserved = reserved + count
      INSERT INTO reservations (...)
      COMMIT
    ELSE:
      ROLLBACK → return "sold out"

OPTIMISTIC LOCKING (alternative):
  Add version column to row
  UPDATE ... WHERE version=X (fails if concurrent update)
  Retry on failure

■ "Use SELECT FOR UPDATE for inventory — prevents double booking
  without complex distributed transactions"
```

E. CONCURRENCY HANDLING

RACE CONDITION SCENARIO:

```
  User A and User B both check: 1 room available
  Both proceed to book → double booking!

SOLUTIONS:
  1. DB-level locking (SELECT FOR UPDATE):
     + Simple, ACID
     - Doesn't scale beyond one DB shard

  2. Optimistic locking (version field):
     + Less blocking, good for low contention
     - Retry logic needed

  3. Redis distributed lock:
     SETNX lock:{hotel}:{date} → atomic
     Hold lock during booking transaction
     + Works across DB shards
     - Redis SPOF, deadlock risk (use TTL)

  4. Message queue + single worker per room:
     Serialize bookings through queue
     + Guaranteed ordering
     - Higher latency

RECOMMENDED: DB pessimistic lock for single DB,
             Redis distributed lock for sharded setup
```

F. SEARCH DESIGN

HOTEL SEARCH:

```
  • Elasticsearch for: location, amenities, text search
  • Filter: price range, star rating, availability

AVAILABILITY in Search:
  Problem: Elasticsearch is eventually consistent
  Don't check exact availability in ES
  ES returns candidate hotels → check inventory in MySQL
  Two-phase: search → filter

CACHING:
  • Cache hotel details (name, photos, amenities) → CDN
  • Cache search results for popular queries (Redis, TTL=5min)
  • Don't cache real-time availability (changes too fast)

PRICING:
  Dynamic pricing based on demand, season, events
  Rules engine: base_price × demand_factor × seasonality
  Store price separately from inventory
```

G. PAYMENT & RELIABILITY

PAYMENT FLOW:

```
  Reservation created (status=PENDING)
  → Payment service → Stripe
  → Payment success → Update status=CONFIRMED
  → Payment failed → status=FAILED, release inventory

IDEMPOTENCY:
  Generate idempotency_key before charging
  If retry: same key → no double charge
  Stripe supports idempotency keys natively

OVERBOOKING (strategic):
  Airlines/hotels intentionally overbook (expect cancellations)
  total_rooms = physical_rooms × 1.05 (5% overbook)
  Handle with upgrades or compensation

CANCELLATION:
  Release inventory immediately
  Refund based on policy (no-refund, partial, full)
  Compensating transaction if payment already processed
```

H. INTERVIEW TIPS

✓ Focus on double-booking prevention (most critical)
✓ Show inventory schema with SELECT FOR UPDATE
✓ Separate search (ES) from inventory check (MySQL)
✓ Address payment idempotency
✓ Mention optimistic vs pessimistic locking trade-offs

■ MUST SAY IN INTERVIEW:
"Use SELECT FOR UPDATE to serialize

```
  inventory decrements. Two-phase: search in ES,
  confirm availability in ACID database."

MNEMONIC: "LOCK-SEARCH-PAY" → Lock-inventory, Elasticsearch-search, Pay-idempotent
```

20 Distributed Email Service

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Send and receive emails at scale (Gmail-scale)
• Components: SMTP, IMAP, spam detection, search
• Key challenges: deliverability, spam, attachments, search
• Scale: 1B users, 100B emails/day (Google scale)

B. REQUIREMENTS

FUNCTIONAL:

• Send email (to/cc/bcc, attachments)
• Receive and store email
• Organize: folders, labels, search
• Spam/virus filtering
• Read receipts, threading

NON-FUNCTIONAL:

• 1B users
• 100B emails/day = 1.16M emails/sec
• Attachment: up to 25MB
• Email persistence (never lose email)
• Search: < 1 sec
• 99.999% availability

C. EMAIL PROTOCOLS

SMTP (Simple Mail Transfer Protocol):

```
    • Sending email (server to server)
    • Port 25 (server-server), 587 (client-server, with TLS)
    • Flow: your SMTP → MX lookup → recipient SMTP

  IMAP (Internet Message Access Protocol):
    • Client reads email from server
    • Emails stay on server
    • Multi-device sync (read on phone = read everywhere)
    • Port 993 (SSL)

  POP3 (Post Office Protocol):
    • Downloads email to device, removes from server
    • Old, single-device use case

  DNS MX Record:
    "To send to user@gmail.com, find MX record of gmail.com"
    MX → mail.google.com → SMTP delivery

  SPF/DKIM/DMARC (anti-spam):
    SPF: "These IPs are authorized to send from this domain"
    DKIM: Cryptographic signature on email headers
    DMARC: Policy for handling SPF/DKIM failures
```

D. HIGH-LEVEL DESIGN

SEND EMAIL:

```
Client → API Server → [Outgoing Queue] → [SMTP Workers]
                                                ↓
                                          DNS MX lookup
                                                ↓
                                     Recipient SMTP server

RECEIVE EMAIL:
External SMTP → [Incoming SMTP Server] → [Filter (spam/virus)]
                                                ↓
                                          [Inbound Queue]
                                                ↓
                                          [Message Store] (S3 + DB)
                                                ↓
                                          [User Mailbox]

ARCHITECTURE:
  SMTP servers: handle delivery/receipt
  API servers: client app communication (REST/IMAP)
  Message store: emails stored (S3 for bodies, MySQL/Cassandra for metadata)
  Search index: Elasticsearch for full-text search
```

E. STORAGE DESIGN

EMAIL BODY: Blob storage (S3)

```
  Path: emails/{user_id}/{email_id}/body.eml
  Attachments: emails/{user_id}/{email_id}/attachments/{filename}
  Dedup: same attachment sent to 10K users → stored once (content hash)

EMAIL METADATA: MySQL/Cassandra
  TABLE: emails
    email_id    UUID        PK
    user_id     BIGINT      FK (recipient)
    thread_id   UUID
    subject     VARCHAR
    from_addr   VARCHAR
    to_addrs    JSONB
    body_s3_key VARCHAR
    is_read     BOOLEAN
    labels      JSONB       ["INBOX", "IMPORTANT"]
    timestamp   TIMESTAMP
    size_bytes  INT

THREADING:
  Thread = group of related emails (reply chain)
  thread_id: hash of original Message-ID
  In-reply-to header links emails to thread

SEARCH:
  Elasticsearch index on: subject, from, body snippet, labels
  Full-body search: index extracted text from body + attachments
  Attachment OCR: extract text from PDFs for search
```

F. DELIVERABILITY & ANTI-SPAM

OUTBOUND DELIVERABILITY:

```
  • Sign emails with DKIM (required by Gmail/Yahoo 2024)
  • Maintain SPF record
  • Warm up new IP addresses (start low volume)
  • Monitor bounce rates (>5% → IP blacklisted)

INBOUND SPAM FILTERING:
  Rule-based: known bad IPs, domain blacklists
  ML-based: content analysis (SpamAssassin, custom model)
  User behavior: "mark as spam" feedback loop
  Reputation score: sender domain/IP

ATTACHMENT SCANNING:
  ClamAV (virus scanner)
  Sandboxing for suspicious files
  Block: .exe, .bat, .ps1 (common malware types)

RATE LIMITING:
  Max 100 emails/day/user (prevent spamming)
  Burst allowed: 500 in first hour for business accounts
```

G. INTERVIEW TIPS

✓ Know SMTP flow (MX record lookup)

✓ Separate storage: S3 for body, DB for metadata

✓ Threading via In-Reply-To header

✓ DKIM/SPF/DMARC for deliverability

✓ Search with Elasticsearch + attachment OCR

■ MUST SAY IN INTERVIEW:

"Email body in S3 (content-addressed for dedup), metadata in DB, full-text search in Elasticsearch, DKIM signing for deliverability"

```
MNEMONIC: "SMTP-STORE-SEARCH-SPAM" → four key pillars
```

21 S3-like Object Storage

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

- Distributed storage for arbitrary binary objects (blobs)
- Examples: Amazon S3, Google GCS, Azure Blob Storage
- Key operations: PUT, GET, DELETE an object
- Not a filesystem (no hierarchy, no update-in-place)
- Immutable objects: write once, read many

B. REQUIREMENTS

FUNCTIONAL:

```
  • PUT /bucket/key → upload object
  • GET /bucket/key → download object
  • DELETE /bucket/key
  • List objects in bucket
  • Versioning, access control (ACL)

NON-FUNCTIONAL:
  • 100PB total storage
  • 99.999999999% (11 nines) durability
  • Objects: 1 byte to 5TB
  • High read throughput (streaming video, downloads)
  • Eventual consistency for list operations
```

C. HIGH-LEVEL DESIGN

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■          API Layer                 ■
■  PUT /bucket/key → [Metadata + Data]  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
         ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■     Metadata Service              ■
■  - bucket/key → object_id mapping     ■
■  - ACL, versioning, tags          ■
■  - Stored in: Cassandra/RDBMS     ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
         ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■     Data Storage Layer            ■
■  - Divide object into chunks (64MB)  ■
■  - Store on distributed storage nodes■
■  - Replicate 3x across AZs/racks    ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

D. DATA PLACEMENT & DURABILITY

ERASURE CODING (S3 approach) ■:

```
  vs. 3x replication:
    3x replication: 200% overhead, 3 copies
    Erasure coding: 33-50% overhead, more durable!

  Reed-Solomon (common):
    Split data into k chunks, create m parity chunks
    Can reconstruct from any k of k+m chunks
    Example: k=6, m=3 → 9 chunks, tolerate 3 failures
    Overhead: 3/6 = 50% (vs 200% for 3x replication)

OBJECT PLACEMENT:
  Consistent hashing → determine storage nodes
  Place on nodes in DIFFERENT racks/AZs (failure domains)
  Primary + 2 replicas (or erasure coded chunks)

DURABILITY CALCULATION:
  MTTF (mean time to failure) per disk = 3 years
  11 nines durability with erasure coding across AZs
  Even if entire AZ down → reconstruct from other AZs

WRITE PATH:
  Client → API → compute object_id (UUID)
  → Write to N storage nodes (parallel)
  → Wait for k successful writes
  → Update metadata
  → Return 200 OK
```

E. METADATA SERVICE

METADATA DB (Cassandra/DynamoDB):

```
  TABLE: objects
    bucket_name  VARCHAR     PARTITION KEY
    object_key   VARCHAR     CLUSTERING KEY
    object_id    UUID        (storage reference)
    size_bytes   BIGINT
    etag         VARCHAR     (MD5 or SHA)
    content_type VARCHAR
    created_at   TIMESTAMP
    version_id   UUID        (if versioning enabled)
    acl          JSONB

OBJECT ID → STORAGE LOCATION:
  object_id → hash → storage nodes (consistent hashing)
  OR: separate "placement service" maps object_id → [node_list]

VERSIONING:
  Same bucket/key → multiple version_id entries
  GET without version → latest version
  DELETE → add delete marker (soft delete)
  Lifecycle policy: delete versions older than N days
```

F. MULTIPART UPLOAD

FOR LARGE OBJECTS (> 100MB):

```
  1. InitiateMultipartUpload → get upload_id
  2. UploadPart: upload parts (5MB to 5GB each)
     Each part → separate storage write
     Returns ETag per part
  3. CompleteMultipartUpload: list of (part_num, etag)
     Server assembles → single object

BENEFITS:
  • Parallel upload (multiple parts simultaneously)
  • Retry only failed parts (not entire file)
  • Resume uploads (save upload_id + completed parts)
  • Mandatory for objects > 5GB

CLEANUP:
  Incomplete multipart uploads waste storage
  Lifecycle rule: abort incomplete uploads after 7 days
```

G. INTERVIEW TIPS

✓ Separate metadata from data storage

✓ Explain erasure coding vs 3x replication

✓ Discuss placement across failure domains (AZ/rack)

✓ Cover multipart upload for large objects

✓ Mention content-addressed storage (dedup by ETag)

■ MUST SAY IN INTERVIEW:

"Erasure coding (Reed-Solomon k+m) for 11-nines durability with 50% overhead vs 200% for replication. Objects are immutable—write once, read many."

```
MNEMONIC: "MECD" → Metadata-separate, Erasure-coding, Chunks, Distributed
```

22 Real-time Leaderboard

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Show ranked list of users/entities by score in real-time
• Examples: Game leaderboards, Stock watchlists, Crypto prices
• Operations: update score, get rank, get top-N, get my rank
• Challenge: millions of score updates/sec + instant rank queries

B. REQUIREMENTS

FUNCTIONAL:

• Update user score (increment or absolute)
• Get top N users with scores and ranks
• Get rank of specific user
• Get users near my rank (±10)

NON-FUNCTIONAL:

• 25M DAU, 2.5M concurrent in tournament
• Score updates: 2.5M/sec (peak)
• Top-100 query: < 10ms
• My rank query: < 500ms
• Leaderboard refreshes visually every sec

C. REDIS SORTED SET ■ (Core Solution)

REDIS SORTED SET (ZSET):

```
  ZADD leaderboard 1500 "user:123"  → O(log N)
  ZINCRBY leaderboard 100 "user:123" → add 100 to score
  ZREVRANK leaderboard "user:123"   → rank (0-indexed) O(log N)
  ZREVRANGE leaderboard 0 99 WITHSCORES → top 100 O(log N + M)
  ZREVRANGEBYSCORE leaderboard +inf -inf WITHSCORES LIMIT 0 100

WHY REDIS SORTED SET?
  • Skip list data structure: O(log N) for all ops
  • Atomic updates (thread-safe INCR)
  • Built-in rank calculation
  • Can hold 500M members (enough for any game)

MEMORY ESTIMATE:
  25M users × ~100 bytes per ZSET entry = 2.5GB RAM
  → Fits in single Redis instance (e.g., r7g.2xlarge)

NEAR-MY-RANK:
  ZREVRANK leaderboard "user:123" → my_rank
  ZREVRANGE leaderboard (my_rank-10) (my_rank+10) WITHSCORES
```

D. ARCHITECTURE

SCORE UPDATE FLOW:

```
  Game Server → [Score Update API] → Redis ZINCRBY
                                          ↓
                                   [Async persist to DB]
                                   (batch write every 30s)

LEADERBOARD QUERY FLOW:
  Client → [API Server] → [Redis ZREVRANGE]
                            ↓ (cache miss for details)
                        [User DB] (get names, avatars)

CACHING STRATEGY:
  Top-100 result: cache in Redis as string with TTL=1s
  For real-time feel: push updates via WebSocket
  Full leaderboard recompute: prohibitively expensive

SCALING:
  Single Redis instance → ~100K writes/sec
  Need 2.5M/sec → Redis Cluster?
  Shard by: leaderboard_id (different games = different shards)
  Within one game: harder to shard (rank spans all users)
```

E. SCALING THE LEADERBOARD

SINGLE LEADERBOARD BOTTLENECK:

All 2.5M users updating same ZSET

```
  Redis single-threaded → ~100K ops/sec per core

SOLUTION: SEGMENTED LEADERBOARD:
  Shard by user_id range: [0-999K], [1M-1.999M], [2M-2.5M]
  Each shard → separate Redis ZSET

  Top-N query:
    Fetch top-N from each shard → merge → global top-N
    Merge N lists of length N → O(N log K) where K=shards

  My Rank:
    1. Query shard containing my user_id → my rank within shard
    2. Query all shards: count users with higher score
    3. Sum → global rank
    Expensive → cache per user or approximate

APPROXIMATE RANK (for very large scale):
  Sample 1% of users → estimate rank
  Or: percentile buckets (top 0.1%, top 1%, top 10%)
```

F. DATABASE PERSISTENCE

TABLE: scores (MySQL)

user_id BIGINT PK

game_id BIGINT PK

score BIGINT

updated_at TIMESTAMP

BATCH WRITE STRATEGY:

```
  Redis → Kafka → DB writer (batch INSERT/UPDATE every 30s)
  On Redis failure: DB is source of truth → rebuild Redis from DB
  Recovery time: 25M users × 1 write/user → few minutes to rebuild

HISTORICAL SCORES:
  TABLE: score_history
    user_id, game_id, score, timestamp (snapshot every hour)
  Enables: "last week's rank" queries
```

G. INTERVIEW TIPS

```
✓ Redis Sorted Set is the answer → know all commands
✓ Discuss O(log N) complexity
✓ Address scaling: shard by leaderboard, merge for global rank
✓ Persist to DB asynchronously (Redis is cache, DB is source of truth)
✓ Near-my-rank feature is interesting edge case

■ MUST SAY: "Redis Sorted Set with ZINCRBY for updates,
  ZREVRANK for my position, ZREVRANGE for top-N.
  O(log N) all ops. Batch persist to DB."

MNEMONIC: "ZSET-RANK-SHARD" → Redis-sorted-set, Rank-O(logN), Shard-by-game
```

23 Payment System

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

- Process payments reliably: accept money, pay out money
- Examples: Stripe, PayPal, Square, Venmo
- Most critical: exactly-once, no money lost, no double charges
- Components: PSP (payment service provider), ledger, reconciliation

B. REQUIREMENTS

FUNCTIONAL:

- Accept payment (credit card, digital wallet)
- Pay out to merchants
- Retry failed payments
- Refunds
- Transaction history

NON-FUNCTIONAL:

- 1M transactions/day = ~12 TPS average, 100 TPS peak
- Exactly-once processing (no double charges)
- 99.99% availability
- Consistency > availability (CP system)
- Audit trail: immutable, every transaction logged
- Latency: < 3 sec for payment response

C. HIGH-LEVEL DESIGN

```
User → [Payment Service] → [PSP: Stripe/Adyen]
                                ↓
                            [Payment Gateway] → Card Networks (Visa/MC)
                                ↓
                            [Issuing Bank] → Approve/Decline

INTERNAL:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■               Payment Service                    ■
■  1. Validate request                            ■
■  2. Create payment order (DB, status=PENDING)■
■  3. Call PSP → get payment_intent              ■
■  4. PSP async webhook → update status          ■
■  5. Update ledger                               ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

D. EXACTLY-ONCE SEMANTICS (CRITICAL) ■

```
PROBLEM: Network retry → PSP called twice → double charge

SOLUTION: IDEMPOTENCY KEYS
  Generate idempotency_key = UUID before calling PSP
  Include key in every API call to PSP
  If same key seen again → return same response (no re-charge)
  PSP stores key → response mapping (with TTL)

  Stripe: `Idempotency-Key: {uuid}` header

OUR SIDE:
  payment_orders table with payment_id (idempotency key)
  UNIQUE constraint on payment_id
  Before calling PSP: INSERT INTO payment_orders(payment_id, ...)
  If INSERT fails (duplicate) → return existing result

STATUS MACHINE:
  PENDING → PROCESSING → SUCCESS/FAILED/REFUNDED

WEBHOOK RELIABILITY:
  PSP calls our webhook (async result)
  Webhook must be idempotent (PSP may retry)
  Store PSP's event_id → process each event_id once
```

E. LEDGER & ACCOUNTING

DOUBLE-ENTRY BOOKKEEPING ■:

```
  Every transaction = two entries (debit + credit)
  Debit one account + Credit another
  Sum of all entries = 0 (always balanced)

  Example: User pays $100:
    DEBIT  accounts_receivable $100  (money owed to us)
    CREDIT merchant_account    $100  (money owed to merchant)

LEDGER TABLE:
  TABLE: ledger (APPEND-ONLY, never update/delete)
    id          BIGINT       PK
    transaction_id UUID
    account_id  BIGINT
    amount      DECIMAL(19,4) (signed: positive=credit, negative=debit)
    currency    VARCHAR(3)
    created_at  TIMESTAMP
    description TEXT

IMMUTABILITY:
  Never UPDATE or DELETE ledger entries
  Corrections via compensating entries (negative amounts)
  Full audit trail preserved forever

RECONCILIATION:
  Daily: compare our ledger with PSP settlement report
  Discrepancies → investigate → manual correction entry
  "PSP is always right" for settlement amounts
```

F. RELIABILITY & FAILURE HANDLING

PSP TIMEOUT:

```
  Call PSP → timeout → is payment charged or not?
  Solution: poll PSP for payment status
  Or: receive async webhook with final status
  Never retry blind → use idempotency key if you retry

DATABASE TRANSACTIONS:
  All payment-related writes in single ACID transaction:
  INSERT payment_order + INSERT ledger_entries → COMMIT together

CIRCUIT BREAKER:
  PSP degraded → open circuit → queue payments → drain when healthy
  Don't fail user immediately if PSP is slow

OUTBOX PATTERN:
  Write to DB (payment_orders + outbox) in one transaction
  Separate process reads outbox → calls PSP → deletes outbox entry
  Guarantees at-least-once PSP call without distributed transaction
```

G. INTERVIEW TIPS

✓ Idempotency key = most important concept for payment systems

✓ Double-entry ledger (append-only) for accounting

✓ Webhook reliability: store event_id, process once

✓ PSP timeout handling: poll for status, don't retry blind

✓ Reconciliation: compare with PSP daily

■ MUST SAY IN INTERVIEW:

"Three pillars: idempotency keys to prevent double charges, double-entry ledger for accounting, reconciliation to catch discrepancies"

```
MNEMONIC: "ILR" → Idempotency, Ledger, Reconciliation
```

24 Digital Wallet

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Store and transfer money between users digitally
• Examples: PayPal, Venmo, Cash App, Google Pay, WeChat Pay
• Key challenge: transfer atomicity (money can't appear or disappear)
• Similar to payment system but with balance management

B. REQUIREMENTS

FUNCTIONAL:

• Top up wallet (from bank/card)
• Transfer money to another user
• Withdraw to bank account
• View balance and transaction history

NON-FUNCTIONAL:

• 1M wallets, 100K transfers/day
• Transfer latency < 1 sec
• Exactly-once: no money created or destroyed
• Strong consistency (CP system, not AP)
• Full audit trail
• ACID guarantees for balance updates

C. BALANCE UPDATE (CRITICAL)

NAIVE APPROACH (WRONG):

UPDATE wallets SET balance = balance - 100 WHERE user_id = A
UPDATE wallets SET balance = balance + 100 WHERE user_id = B

```
  Problem: if crash between two updates → money disappears!

CORRECT APPROACH: DATABASE TRANSACTION
  BEGIN TRANSACTION
    SELECT balance FROM wallets WHERE user_id = A FOR UPDATE  -- lock
    IF balance >= 100:
      UPDATE wallets SET balance = balance - 100 WHERE user_id = A
      UPDATE wallets SET balance = balance + 100 WHERE user_id = B
      INSERT INTO transactions (from, to, amount, ...)
      COMMIT
    ELSE:
      ROLLBACK -- insufficient funds

DOUBLE-ENTRY LEDGER (same as payment system):
  DEBIT  user_A_account  $100
  CREDIT user_B_account  $100
  → Ledger always balanced, full audit trail

CONCURRENT TRANSFERS:
  A→B while B→A simultaneously
  DEADLOCK risk if locking in different order!
  Fix: always lock accounts in ascending order of user_id
```

D. DISTRIBUTED WALLET (SCALING)

```
PROBLEM: Single DB → bottleneck at scale

HORIZONTAL PARTITIONING:
  Shard by user_id → each shard owns subset of wallets
  A→B transfer: if A and B on SAME shard → local ACID transaction
  A→B transfer: if A and B on DIFFERENT shards → distributed transaction!

DISTRIBUTED TRANSACTION OPTIONS:
  1. 2PC (Two-Phase Commit):
     Phase 1: Prepare (ask both shards to lock + prepare)
     Phase 2: Commit (both shards commit)
     Problem: coordinator failure → stuck in prepared state

  2. SAGA PATTERN:
     Step 1: Debit A (local transaction on shard1)
     Step 2: Credit B (local transaction on shard2)
     If step 2 fails → compensating transaction: credit A back
     Eventual consistency: temporary inconsistency between steps

  3. Event Sourcing + Outbox:
     Write events to outbox → process asynchronously
     Guarantees at-least-once + idempotent apply

■ "Prefer Saga for distributed transfers.
  Compensating transactions are the rollback mechanism."
```

E. DATABASE DESIGN

TABLE: wallets

wallet_id BIGINT PK

user_id BIGINT UNIQUE FK

```
  balance     DECIMAL(19,4)  -- never negative
  currency    VARCHAR(3)
  updated_at  TIMESTAMP
  version     BIGINT  -- optimistic locking

TABLE: wallet_transactions
  txn_id      UUID        PK
  from_wallet BIGINT      FK
  to_wallet   BIGINT      FK
  amount      DECIMAL(19,4)
  status      ENUM        (PENDING/COMPLETED/FAILED)
  created_at  TIMESTAMP
  idempotency_key UUID    UNIQUE

TABLE: ledger (append-only, double-entry)
  entry_id    BIGINT      PK
  txn_id      UUID        FK
  wallet_id   BIGINT
  entry_type  ENUM        (DEBIT/CREDIT)
  amount      DECIMAL(19,4)
  balance_after DECIMAL(19,4)
  created_at  TIMESTAMP
```

F. INTERVIEW TIPS

✓ Always use DB transactions for balance updates

✓ Deadlock prevention: lock wallets in ascending ID order

✓ Double-entry ledger for auditability

✓ Distributed transfers: Saga pattern with compensation

✓ Idempotency key on transfers (prevent double debit)

■ MUST SAY IN INTERVIEW:

"Money transfer = double-entry bookkeeping

```
  in ACID transaction. For cross-shard: Saga pattern
  with compensating transactions."

COMPARE WITH PAYMENT SYSTEM:
  Payment: external money flow (card → merchant)
  Wallet: internal money flow (user A → user B)
  Both: need idempotency, ledger, exactly-once

MNEMONIC: "ACID-SAGA-LEDGER" → core concepts
```

25 Stock Exchange

System Design Interview Notes | SDE-2 Level | ByteByteGo

A. PROBLEM UNDERSTANDING

• Electronic trading platform matching buy/sell orders
• Examples: NYSE, NASDAQ, Binance (crypto)
• Hardest interview problem: ultra-low latency + financial correctness
• Key component: Order Matching Engine
• Latency requirements: sub-millisecond for matching!

B. REQUIREMENTS

FUNCTIONAL:

• Place order (market, limit, stop)
• Cancel order

```
  • Order matching (buy meets sell → trade)
  • View order book (all open orders)
  • Historical trades (ticker data)

NON-FUNCTIONAL:
  • Throughput: 100K orders/sec
  • Matching latency: < 1ms (critical!)
  • Exactly-once order processing (financial integrity)
  • Full audit trail
  • Market data: real-time price feed to all clients
  • 99.999% availability during market hours
```

C. ORDER TYPES

MARKET ORDER: Execute immediately at best price

```
    Buy 100 AAPL at market → fill at whatever price available
    Risk: price impact in illiquid markets

  LIMIT ORDER: Execute only at specified price or better
    Buy 100 AAPL at $180 or less
    Goes into order book if not immediately fillable
    Rests until filled or cancelled

  STOP ORDER: Trigger market order when price reaches stop
    Stop-loss: sell if price drops to $170 (limit loss)

  ORDER BOOK:
    BUY SIDE (BIDS):     SELL SIDE (ASKS):
    Price    Qty         Price    Qty
    $181.00  500   ←→    $181.50  300
    $180.50  1000         $182.00  800
    $180.00  2000         $183.00  1500

    Spread = Ask - Bid = $181.50 - $181.00 = $0.50
    Trade occurs when bid price >= ask price
```

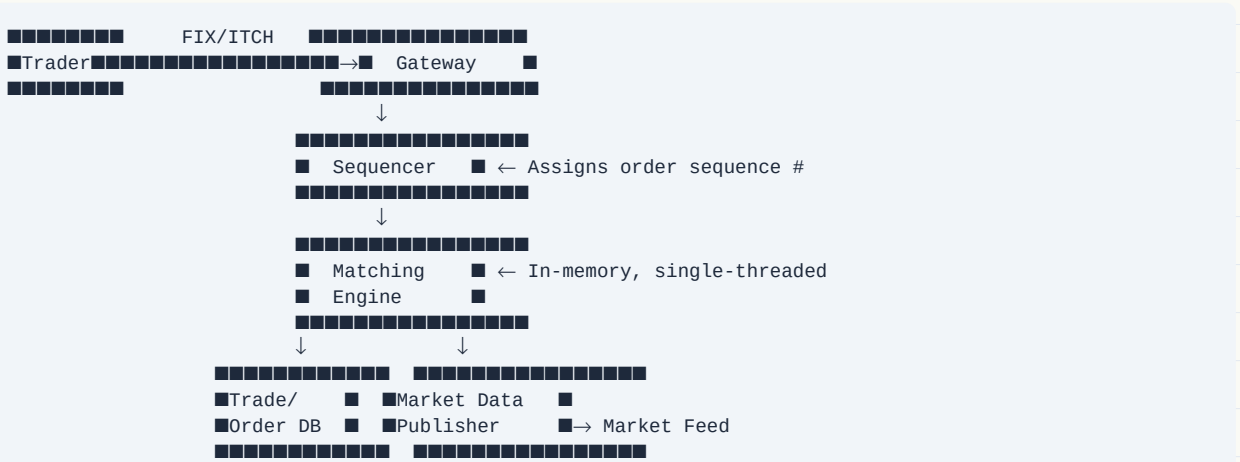
D. MATCHING ENGINE (CORE) ■

PRICE-TIME PRIORITY:

```
  1. Best price first (highest bid, lowest ask)
  2. Ties: earliest order gets priority (FIFO)

DATA STRUCTURE: Red-Black Tree or Skip List per side
  O(log N) insertion, O(1) best price access
  Or: Priority Queue / TreeMap<Price, Queue<Order>>

MATCHING ALGORITHM:
  New BUY order arrives (limit $182):
    Find best ask: $181.50 (available in ask side)
    $182 >= $181.50 → MATCH!
    Trade at $181.50 (ask price, price improvement for buyer)
    Reduce quantities, generate trade event
    Continue matching until order filled or no more matches

CRITICAL: Single-threaded matching engine
  Why? Avoid concurrency complexity, deterministic, auditable
  Ultra-low latency via in-memory data structures
  No DB writes in critical path → write to journal/WAL after match

SEQUENCER:
  All orders globally ordered before entering matching engine
  Sequencer assigns monotonic sequence number
  Matching engine processes in strict sequence order
  → Deterministic, replayable
```

E. HIGH-LEVEL DESIGN

CLIENT EXCHANGE




```
PROTOCOLS:
  FIX: industry standard order entry protocol
  ITCH/OUCH: NASDAQ's proprietary low-latency protocol
  WebSocket: for retail clients
```

F. MARKET DATA DISTRIBUTION

AFTER TRADE:

```
  Matching engine → publish trade event
  Market Data Publisher → broadcast to all subscribers
  Price feed: bid, ask, last price, volume

MARKET DATA CONSUMERS:
  • Other traders (prices)
  • Risk systems (position monitoring)
  • Regulatory reporting
  • Index calculators

DISTRIBUTION:
  Multicast UDP: lowest latency (no TCP overhead)
  Guaranteed delivery: sequence numbers + retransmit on request
  Snapshot + incremental updates

ORDER BOOK AGGREGATION:
  Full depth (all orders) vs Level 1 (best bid/ask only)
  Level 2: top 5-10 price levels
  Exchanges sell market data feeds (revenue stream)
```

G. RISK MANAGEMENT

PRE-TRADE RISK CHECKS (before entering matching engine):

```
  • Position limits (can't buy more than X shares)
  • Capital checks (sufficient balance)
  • Price sanity checks (order price within ±X% of market)
  • Circuit breakers (halt trading if price moves >10% in 5 min)

POST-TRADE:
  • Clearing: ensure both parties can settle
  • Settlement: T+2 for equities (two days after trade)
  • CCP (Central Counterparty Clearing): assumes counterparty risk

KILL SWITCH:
  Emergency stop for runaway algorithms (Flash Crash 2010)
  Immediately cancel all open orders
  Must execute in < 1ms
```

H. FAULT TOLERANCE

MATCHING ENGINE CRASH:

```
  Hot standby: exact replica of primary
  Sequencer → replicate to standby in real-time
  Standby can take over in <1 sec

STATE RECOVERY:
  Journal/WAL: every order logged before matching
  Replay journal → reconstruct order book state
  Deterministic: same inputs → same state (from sequencer)

TESTING:
  Simulation mode: test without real money
  Shadow mode: process real orders but don't execute
  Market replay: replay historical data to test engine
```

I. INTERVIEW TIPS

✓ Focus on matching engine: single-threaded, in-memory

✓ Price-time priority algorithm

✓ Sequencer for deterministic ordering (enable replay)

✓ No DB writes in hot path → WAL async
✓ Hot standby for HA, journal for recovery

■ MUST SAY: "Matching engine is single-threaded, in-memory
  for sub-millisecond latency. Sequencer provides ordering.
  Everything after match is async (WAL, market data, DB write)."

COMMON MISTAKES:
  ✗ Suggesting microservices with DB calls in matching path
  ✗ Not knowing order types (market, limit, stop)
  ✗ Ignoring sequencer for determinism

MNEMONIC: "SMASH" → Sequencer, Matching-engine, Async-WAL,
  Standby-hot, Hot-path-no-DB